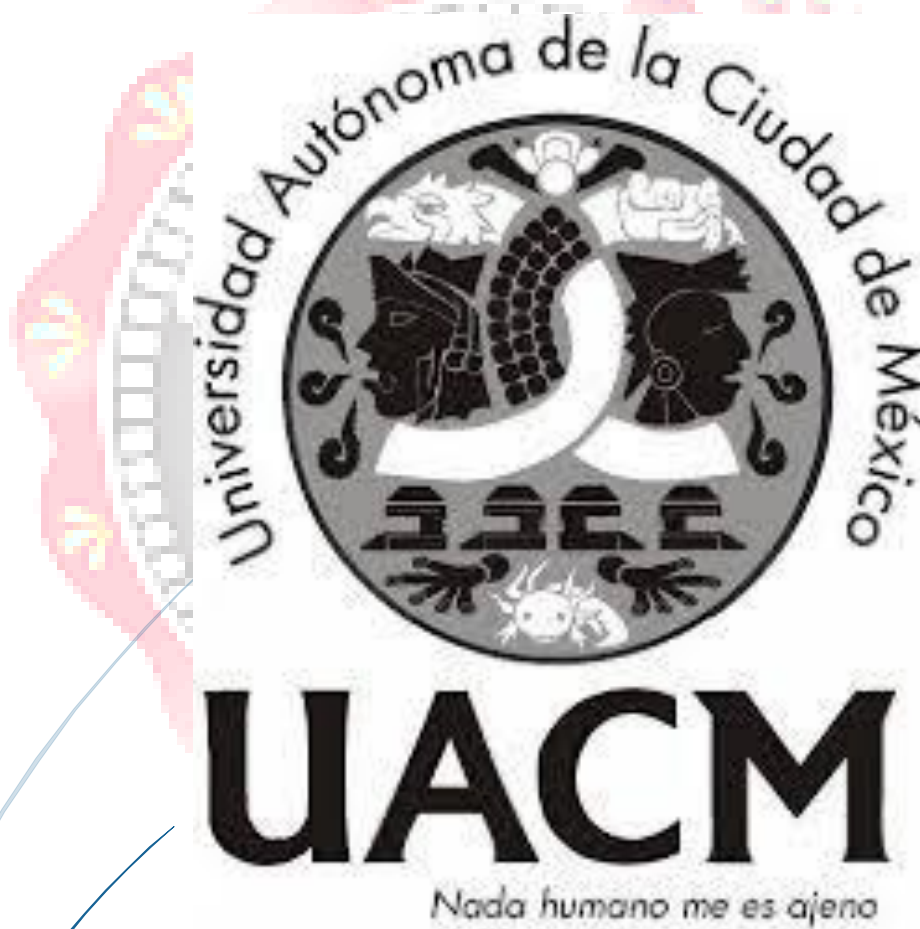


17-10-2025

# Diseño de la arquitectura del software



Alejandro Viveros Hernández -20-011-0828  
Andres Hernández Gutierrez - 17-011-0053  
UNIVERSIDAD AUTONOMA DE LA CIUDAD DE MEXICO

# Índice

## Contenido

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Historial de versiones .....                                                             | 2  |
| Diagrama de Gantt inicial .....                                                          | 3  |
| 1.- Introducción .....                                                                   | 3  |
| 1.1 Objetivo .....                                                                       | 5  |
| 1.2 Alcance .....                                                                        | 5  |
| 2. Identificación de los stakeholders .....                                              | 6  |
| 2.1 Preocupaciones y anti preocupaciones .....                                           | 9  |
| 3.-Viewpoints de Arquitectura .....                                                      | 12 |
| 3.1 Context Viewpoint – Proyecto: Historia Interactiva IA.....                           | 12 |
| 3.2. Composition Viewpoint (Punto de Vista de Composición) .....                         | 15 |
| 3.3. Logical Viewpoint (Punto de Vista Lógico) .....                                     | 18 |
| 3.4. Interaction Viewpoint (Punto de Vista de Interacción) .....                         | 22 |
| 3.5. Deployment Viewpoint (Punto de Vista de Despliegue) .....                           | 25 |
| 3.6. Data Viewpoint (Punto de Vista de Datos) .....                                      | 28 |
| 3.7. Architectural Patterns Viewpoint (Punto de Vista de Patrones Arquitectónicos) ..... | 31 |
| 4. Vistas de la Arquitectura .....                                                       | 34 |
| 4.1. Vista de Contexto del Sistema "Historia Interactiva IA (IotopIA)" .....             | 35 |
| 4.2. Vista de Composición del Sistema ( Estructura) .....                                | 39 |
| 4.3. Vista Lógica del Sistema "Historia Interactiva IA" .....                            | 42 |
| 4.4. Vista de Interacción del Sistema "Historia Interactiva IA" .....                    | 48 |
| 4.5. Vista de Despliegue del Sistema "Historia Interactiva IA" .....                     | 54 |
| 4.6. Vista de Datos del Sistema "Historia Interactiva IA" .....                          | 57 |
| 4.7. Vista de Patrones de Diseño y Arquitectura.....                                     | 60 |
| 5. Coherencia y Reglas de Correspondencia (Cláusula 5.7) .....                           | 63 |
| 6. Justificación de las Decisiones Arquitectónicas (Architecture Rationale) .....        | 65 |
| Conclusión .....                                                                         | 69 |

# Historial de versiones

| Version 1.0                 |           |                                                                       |
|-----------------------------|-----------|-----------------------------------------------------------------------|
| Nombre:                     | Fecha     | ¿Que se Documentoe implemento?                                        |
| Alejandro Viveros Hernández | 20/9/2025 | Generacion del documento inicial<br>Análisis de implemenetcion previa |
| Andres Gutierrez Hernandez  | 20/9/2025 | Generacion de los diagramas<br>iniciales                              |

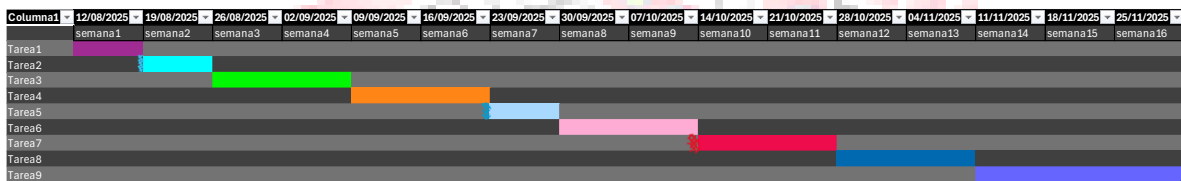
| Version 1.0                 |           |                                                               |
|-----------------------------|-----------|---------------------------------------------------------------|
| Nombre:                     | Fecha     | ¿Que se Documentoe implemento?                                |
| Alejandro Viveros Hernández | 1/10/2025 | Generacion de la seccion 1 y 2<br>estructuración del proyecto |
| Andres Gutierrez Hernandez  | 2/9/2025  | Generacion de los diagramas<br>análisis de base de datos      |

| Version 1.0                 |            |                                                                               |
|-----------------------------|------------|-------------------------------------------------------------------------------|
| Nombre:                     | Fecha      | ¿Que se Documentoe implemento?                                                |
| Alejandro Viveros Hernández | 30/10/2025 | Generacion de la seccion 3 y 4<br>implemenetacion de clases en el<br>proyecto |
| Andres Gutierrez Hernandez  | 30/10/2025 | Generacion de los diagramas de<br>para seccion 4 implementacion<br>registro   |

| Version 1.0                 |            |                                                                       |
|-----------------------------|------------|-----------------------------------------------------------------------|
| Nombre:                     | Fecha      | ¿Que se Documentoe implemento?                                        |
| Alejandro Viveros Hernández | 20/11/2025 | Generacion del documento final<br>seccion 5 y 6 revision y ajuste del |

|                            |            |                                                                      |
|----------------------------|------------|----------------------------------------------------------------------|
|                            |            | edocuemnto seccion1,2,3,4                                            |
|                            |            | implemenetacion de clases intento de depuración del proyecto fallida |
| Andres Gutierrez Hernandez | 20/11/2025 | Implementación base de datos inicio de seision                       |

## Diagrama de Gantt inicial



No se ha podido realizar la documentación e implementación en tiempo y forma.  
motivos :

- cambio de personal de la empresa
- complicaciones en el proyecto
- ajustes de documentacion

## 1.- Introducción

El presente documento describe la arquitectura del sistema “Historia Interactiva IA”, una plataforma orientada a la generación dinámica de narrativas mediante técnicas de inteligencia artificial donde las decisiones del usuario determinan el curso y la dificultad de la historia. La aplicación ofrece una experiencia interactiva en la que el lector deja de ser un mero observador y participa activamente en la construcción del relato: tras una breve personalización inicial, el motor de IA genera escenas y propone al menos tres opciones significativas en cada punto de decisión; las elecciones del usuario no solo condicionan la trama sino que alteran el grado de reto

de las situaciones futuras mediante un mecanismo de adaptación de dificultad. Además de la experiencia narrativa en tiempo real, la plataforma incorpora funcionalidades de gestión de cuentas, persistencia de historias para reanudación posterior y un panel de progreso que registra los contextos clave y las decisiones tomadas, permitiendo al usuario revisar y continuar su viaje narrativo cuando lo desee.

Desde la perspectiva organizativa y técnica, este documento sitúa el sistema en su entorno: especifica los límites del producto, identifica los actores principales implicados en su uso y operación, y clarifica las interacciones con servicios externos como sistemas de autenticación, almacenes de datos y posibles servicios de IA en la nube. Está pensado como una guía viva para desarrolladores, diseñadores, evaluadores y administradores, de forma que todos los participantes del proyecto compartan una visión coherente sobre las responsabilidades funcionales, las dependencias tecnológicas y las expectativas no funcionales. El enfoque de la arquitectura busca equilibrar la flexibilidad necesaria para iterar sobre los modelos de generación y ajuste de dificultad con la solidez requerida para proteger datos de usuarios, garantizar tiempos de respuesta razonables y facilitar el mantenimiento y la evolución del producto.

La justificación de esta arquitectura radica en cuatro necesidades concretas: primero, proporcionar una experiencia narrativa coherente y adaptativa que mantenga al usuario comprometido; segundo, permitir que el equipo de desarrollo implemente y despliegue componentes de IA y almacenamiento de forma modular y trazable; tercero, ofrecer mecanismos de seguridad y protección de datos que cumplan con buenas prácticas; y cuarto, dar soporte a la evolución del sistema para incorporar nuevos géneros, mejoras en el motor de IA o integración con terceros. Para lograrlo, la documentación arquitectónica que aquí se presenta adopta un enfoque basado en viewpoints conforme a la norma ISO/IEC/IEEE 42010, de modo que cada punto de vista —contexto, composición, lógico, de interfaz, de estructura, de interacción, de dinámica de estados y de algoritmo— quede claramente definido, con sus modelos, reglas de correspondencia y métodos de análisis.

Finalmente, este documento se concibe como un instrumento de comunicación técnica y de gobernanza: permitirá validar que las decisiones de diseño satisfacen los requisitos funcionales y de calidad, servirá como referencia para la implementación y las pruebas, y facilitará la toma de decisiones futuras sobre escalabilidad, despliegue y mantenimiento. Su estructura modular y su orientación a viewpoints facilitan la actualización incremental de la arquitectura a medida que el

proyecto avance y surjan nuevas necesidades, sin perder la trazabilidad entre requisitos, diseño y entrega.

## 1.1 Objetivo

El objetivo de este documento es definir, explicar y formalizar la arquitectura del sistema “Historia Interactiva IA” de tal manera que sirva como referencia única y autoritativa para todos los actores del proyecto: describe la estructura del sistema, las relaciones entre sus componentes, los modelos de interacción con usuarios y servicios externos, las reglas de correspondencia entre vistas y los criterios de calidad que deben cumplirse, con el fin de guiar el desarrollo, el despliegue y la evolución del producto. A través de una documentación basada en viewpoints conforme al anexo B de la norma ISO/IEC/IEEE 42010, se pretende proporcionar una base clara para la implementación técnica, facilitar la verificación y validación frente a los requisitos y asegurar que las decisiones arquitectónicas soporten la adaptabilidad narrativa, la seguridad de los datos, el rendimiento operativo y la facilidad de mantenimiento del sistema a lo largo del tiempo.

## 1.2 Alcance

El presente documento de arquitectura cubre el diseño y la estructura del sistema “Historia Interactiva IA”. El alcance del sistema se define por las siguientes funcionalidades y límites:

Dentro del Alcance (In Scope):

- **Gestión de Cuentas de Usuario:** El sistema permitirá el registro, autenticación (inicio de sesión) y cierre de sesión de los usuarios.
- **Personalización de la Narrativa:** Los usuarios podrán definir preferencias iniciales, como el género o el tono, que el motor de IA utilizará para generar la historia.
- **Generación Dinámica de Historias:** El núcleo del sistema es un motor de IA que creará escenas narrativas y presentará al usuario al menos tres opciones de decisión significativas en puntos clave.
- **Adaptación de Dificultad:** Las elecciones del usuario influirán en el nivel de reto de las situaciones futuras, ajustando la dificultad de la narrativa de forma dinámica.

- **Persistencia y Progreso:** El sistema guardará el progreso de las historias iniciadas, permitiendo a los usuarios pausar y reanudar sus partidas más tarde. Se registrarán las decisiones clave tomadas.
- **Interacción con Servicios Externos:** La arquitectura definirá las interacciones con un sistema de autenticación, una base de datos para la persistencia y el motor de IA narrativa.

#### Fuera del Alcance (Out of Scope):

- **Creación de Contenido por el Usuario:** El sistema no permitirá que los usuarios escriban o aporten su propio contenido a las narrativas; únicamente podrán elegir entre las opciones generadas por la IA.

## 2. Identificación de los stakeholders

Los stakeholders identificados son los siguientes:

| Stakeholders            | Descripción                                                                                                                                                                                                                                                                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Usuarios Finales</i> | <p>Es la persona que interactúa directamente con la aplicación para vivir la experiencia narrativa. Su satisfacción es el objetivo principal del producto.</p> <p>Intereses Clave:</p> <ul style="list-style-type: none"> <li>• <b>Experiencia Fluida:</b> Desea una interfaz intuitiva y tiempos de respuesta rápidos.</li> </ul> |

- Coherencia Narrativa: Espera que la historia generada por la IA sea lógica y que sus decisiones tengan impacto.

- Privacidad y Seguridad: Necesita certeza de que su información se maneja confidencialmente.

- Adaptabilidad: Busca un reto equilibrado que se ajuste a su estilo de juego.

### *Desarrolladores*

Es el miembro del equipo técnico responsable de construir, mantener y evolucionar el sistema basándose en la arquitectura definida.

Intereses Clave:

- Claridad Arquitectónica: Necesita documentación clara de componentes y responsabilidades.

- Modularidad: Le interesa el desacoplamiento (especialmente el motor de IA) para facilitar actualizaciones.

- Mantenibilidad: Busca facilitar la depuración y la adición de nuevos géneros narrativos.



*Administradores  
del Sistema*

- Integración Sencilla: Requiere interfaces bien definidas con la BD y la API externa.

Es el encargado de la infraestructura operativa, asegurando que el servidor web, la base de datos y la conexión con la API externa funcionen correctamente.

Intereses Clave:

- Disponibilidad: Necesita que el sistema sea estable y se recupere ante fallos de la API de IA.
- Monitoreo y Logs: Requiere herramientas para rastrear errores en tiempo real (ej. fallos de conexión).
- Gestión de Recursos: Le interesa la eficiencia de las consultas a la BD y el consumo de ancho de banda.
- Seguridad: Prioriza la protección contra ataques (SQL Injection, XSS) y el resguardo de credenciales.

*Cliente /  
Patrocinador*

Es la entidad o persona que define el alcance del proyecto, establece los requisitos de negocio y financia los recursos (como los créditos de la API de IA).

### Intereses Clave:

- Viabilidad Económica: Le preocupa el control de costos operativos (ej. consumo de tokens de OpenAI).
- Cumplimiento de Requisitos: Busca que el producto final coincida con la visión original.
- Escalabilidad: Le interesa que el sistema soporte múltiples usuarios concurrentes sin degradar el servicio.
- Retención: Busca que la calidad narrativa sea suficiente para mantener el interés del usuario.

## 2.1 Preocupaciones y anti preocupaciones

A continuación, se detallan las preocupaciones clave para los diferentes stakeholders del sistema, que han servido de guía para las decisiones arquitectónicas. Estas preocupaciones se abordarán a través de las diferentes vistas de la arquitectura.

Preocupaciones y antipreocupaciones de los stakeholders

| <b>Stakeholder</b>               | <b>Descripción</b>                                                                        | <b>Preocupaciones</b>                                                                                                                                                                                                                                                                                                                              | <b>Anti-preocupaciones</b>                                                                                                                                                                                               |
|----------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Usuario Final</i>             | Son el foco principal del producto. Su experiencia de uso determina el éxito del sistema. | <p>Interactividad: La experiencia debe ser fluida, con alta capacidad de respuesta.</p> <p>Coherencia de la historia: El motor de IA debe generar narrativas lógicas sin saltos argumentales.</p> <p>Adaptabilidad: La dificultad debe ajustarse dinámicamente sin frustrar ni aburrir.</p> <p>Privacidad: Sus datos deben ser confidenciales.</p> | <p>Detalles técnicos: No le interesa qué tecnología de base de datos se usa ni qué patrón de diseño se implementó, siempre que el juego funcione.</p>                                                                    |
| <i>Desarrollador</i>             | Responsables de implementar, mantener y evolucionar el sistema.                           | <p>Entendimiento del sistema: Claridad en las interacciones con servicios externos.</p> <p>Modularidad: Desacoplamiento de la IA para permitir mejoras.</p> <p>Evolución: Flexibilidad para añadir nuevos géneros y modelos de IA.</p>                                                                                                             | <p>Algoritmos internos de la IA: No se detallará cómo funciona la red neuronal de GPT internamente, solo su interfaz.</p> <p>Código de bajo nivel: No se documenta cada línea de código, sino la estructura general.</p> |
| <i>Administrador del Sistema</i> | Responsables de la operación y el despliegue.                                             | <p>Disponibilidad (Uptime): El sistema debe manejar los tiempos de</p>                                                                                                                                                                                                                                                                             | <p>Lógica Narrativa: No le preocupa si la historia tiene sentido literario o si</p>                                                                                                                                      |

Cliente

|                                                |                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | <p>inactividad de la API externa sin "colgarse".</p> <p>Despliegue: La instalación en el servidor (Apache/PHP/Postgres) debe ser replicable y sencilla.</p> <p>Logs de Error: Debe haber registro de fallos críticos para auditoría.</p>                                                                | <p>el juego es divertido, solo que los servicios no fallen.</p> <p>Diseño UI: No le afectan los cambios estéticos en el frontend.</p>                                                                                          |
| <p>Responsable del negocio y los recursos.</p> | <p>Control de Costos: La arquitectura debe optimizar las llamadas a la API para no desperdiciar tokens/dinero.</p> <p>Escalabilidad: El sistema debe soportar crecimiento en el número de usuarios.</p> <p>Plazos: La arquitectura debe permitir un desarrollo ágil para cumplir fechas de entrega.</p> | <p>Patrones de Diseño: No le interesa si se usa MVC o State Pattern, siempre que el producto sea mantenible y rentable.</p> <p>Tecnología específica: Le es indiferente el lenguaje (PHP), mientras cumpla los requisitos.</p> |

### 3.-Viewpoints de Arquitectura

Esta sección define los puntos de vista (viewpoints) que se utilizarán para describir la arquitectura del sistema "Historia Interactiva IA". Cada viewpoint está documentado conforme a la norma ISO/IEC/IEEE 42010 para asegurar su claridad y consistencia.

**!!!Atención!!! los modelos utilizados en esta sección son meramente representativos, se utilizará y definirá el modelo definido en esta sección en la sección 4 (Vistas de la arquitectura), con el proyecto ya involucrado.**

#### 3.1 Context Viewpoint – Proyecto: Historia Interactiva IA

##### Viewpoint name

Context

Viewpoint

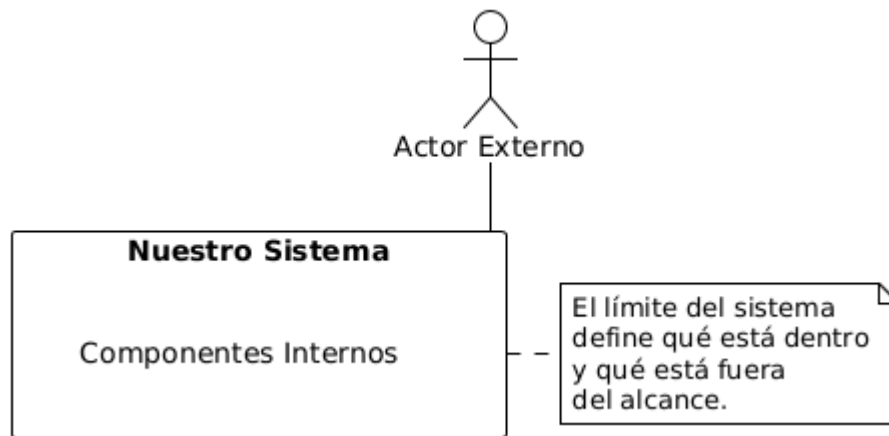
El Punto de Vista del Contexto. Se centra en mostrar el sistema como una “caja negra” dentro de su entorno, resaltando las relaciones con actores humanos, sistemas externos y servicios de soporte.

##### resumen

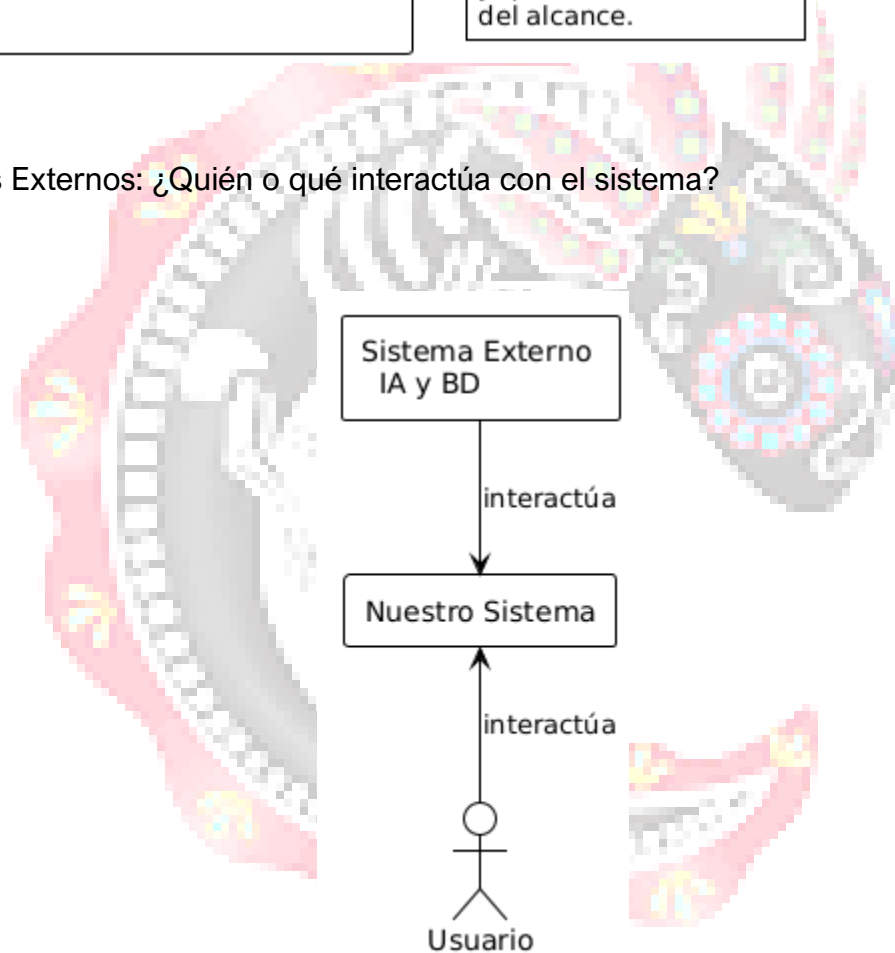
El Context Viewpoint se encarga de describir el sistema de manera global, identificando los límites que lo separan de su entorno, los actores externos con los que interactúa y los principales flujos de información que se intercambian. En el caso

##### Preocupaciones que Aborda (Concerns):

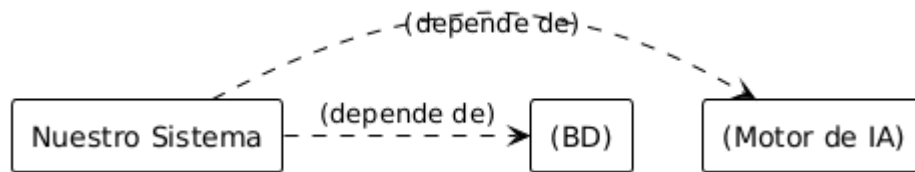
Límites del Sistema: ¿Dónde empieza y termina el sistema?



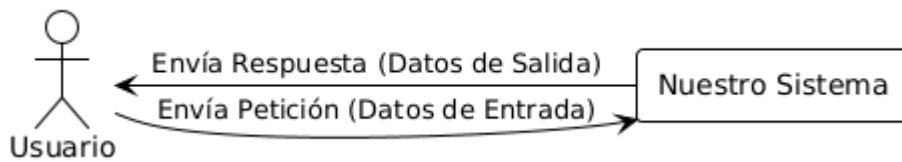
Actores Externos: ¿Quién o qué interactúa con el sistema?



Dependencias Externas: ¿De qué otros sistemas o servicios depende?



Flujos de Información Principales: ¿Qué datos clave se intercambian con el exterior?



## Stakeholders Típicos:

**Desarrolladores:** Para comprender las interacciones con servicios externos.  
Para trazar los requisitos y validar las interacciones externas.

**Cliente:** Para evaluar el alcance funcional y la viabilidad del producto.

**Administradores del Sistema:** Para entender las dependencias operativas.  
Para trazar los requisitos y validar las interacciones externas.

## Tipos de Modelos (Model Kinds):

**Diagrama de Contexto del Sistema:** Un diagrama que visualiza de manera simple las fronteras del sistema y las interacciones principales con su entorno.

**Tablas de Relaciones:** Listados detallados que describen cada actor externo y los datos que intercambia con el sistema.

## Operaciones sobre las Vistas:

**Creación:** Se construye a partir de los requisitos funcionales y entrevistas con stakeholders.

**Interpretación:** Permite a los interesados entender el rol del sistema y cómo se conecta con su entorno.

**Análisis:** Se utiliza para verificar que no se ha omitido ningún actor clave y que los flujos de información están bien delimitados.

**Diseño/Implementación:** Sirve como base para definir las interfaces de comunicación y las dependencias externas a implementar.

## 3.2. Composition Viewpoint (Punto de Vista de Composición)

### Nombre del Viewpoint:

Composition Viewpoint (Punto de Vista de Composición).

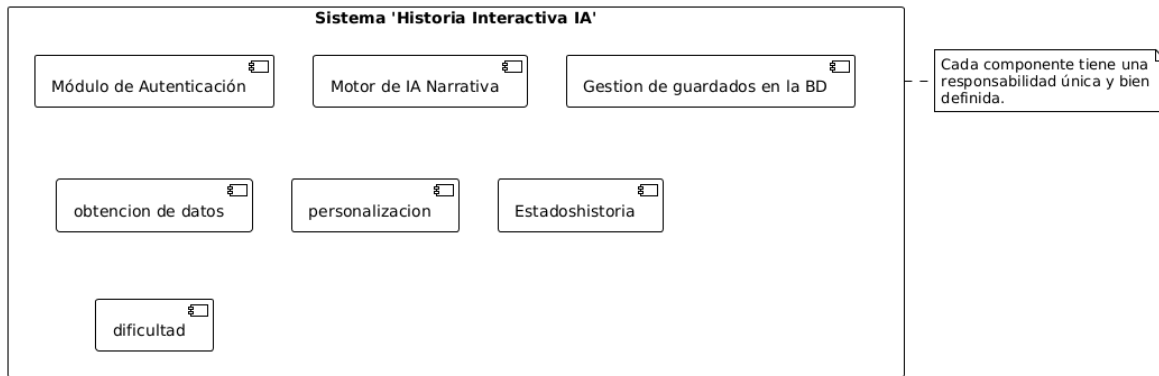
### Resumen:

Este punto de vista se centra en la estructura interna del sistema y en cómo sus principales componentes se organizan para cumplir con la funcionalidad global. Permite entender de qué partes está formado el sistema, cómo se relacionan y cuáles son sus responsabilidades. Este viewpoint es esencial para identificar los bloques de construcción de la aplicación, sus límites y los flujos de información entre ellos.

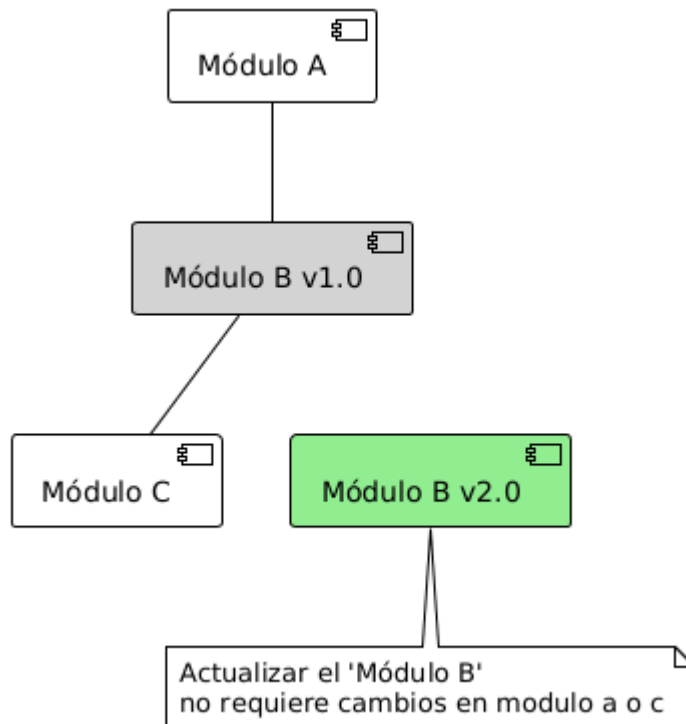


## Preocupaciones que Aborda (Concerns):

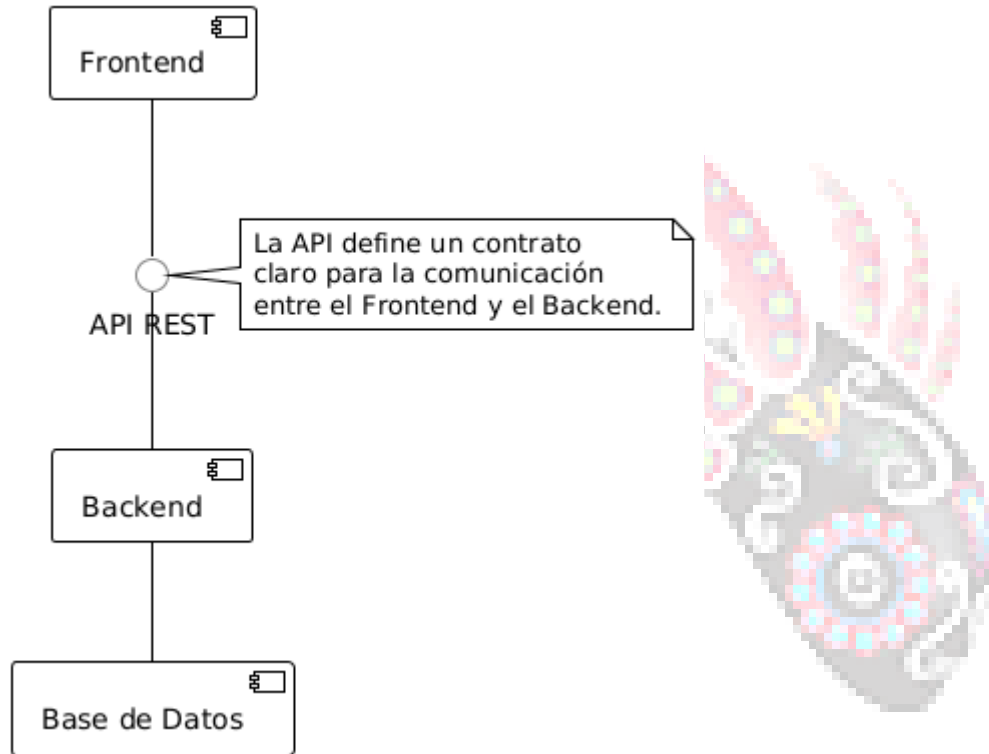
- Modularidad: Que el sistema esté dividido en componentes con responsabilidades claras y poco acoplamiento.



Mantenibilidad: Que cada componente sea fácilmente actualizable de forma independiente.



Integración: Que los diferentes módulos (IA, base de datos, autenticación, etc.) trabajen de manera conjunta sin conflictos.



Anti-preocupaciones: Este viewpoint no detalla los algoritmos internos de la IA ni las interacciones paso a paso con el usuario.

### Stakeholders Típicos:

Desarrolladores: Para comprender la estructura modular al implementar, probar y mantener cada componente.

Para conocer los módulos y sus límites al diseñar casos de prueba de integración.

Administradores de Sistema: Para saber cómo se despliegan los módulos en la infraestructura.

## **Tipos de Modelos (Model Kinds):**

Diagrama de Componentes UML: Para visualizar cómo se descompone el sistema en componentes internos, sus interfaces y conexiones.

## **Operaciones sobre las Vistas:**

Creación: Se construye a partir de la identificación de funcionalidades críticas como autenticación, generación de historia y persistencia de datos.

Interpretación: Permite a los interesados entender cómo se organiza internamente la aplicación.

Análisis: Facilita la identificación de redundancias, acoplamientos innecesarios o dependencias críticas.

Diseño/Implementación: Actúa como una guía directa para la construcción de la arquitectura de software.

## **Fuentes:**

ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description.

Documentación del proyecto “Historia Interactiva IA”.

## **3.3. Logical Viewpoint (Punto de Vista Lógico)**

### **Nombre del Viewpoint:**

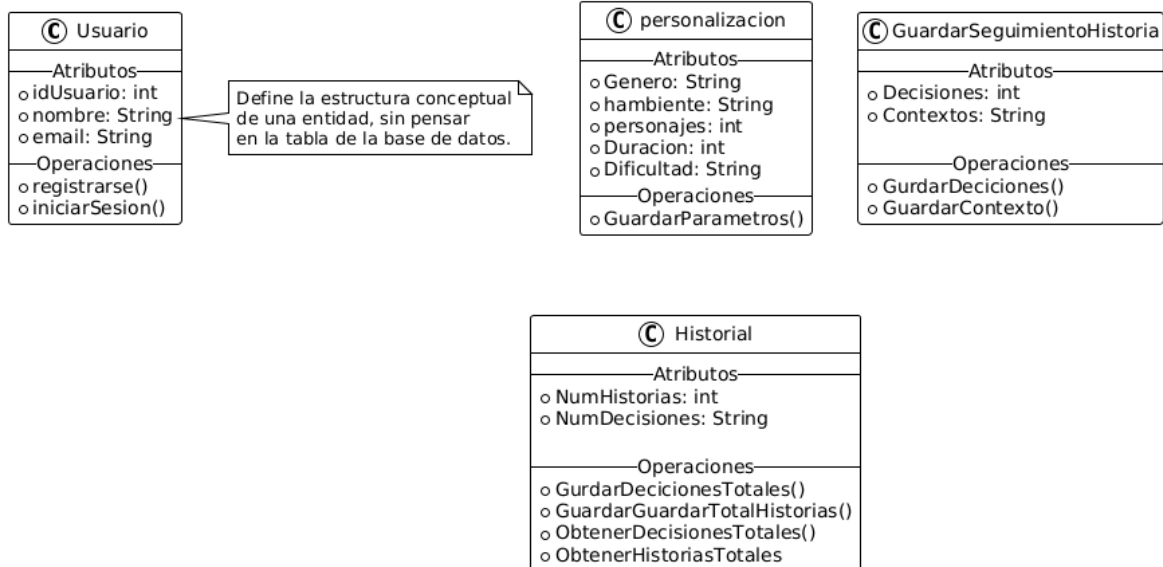
Logical Viewpoint (Punto de Vista Lógico).

## **Resumen:**

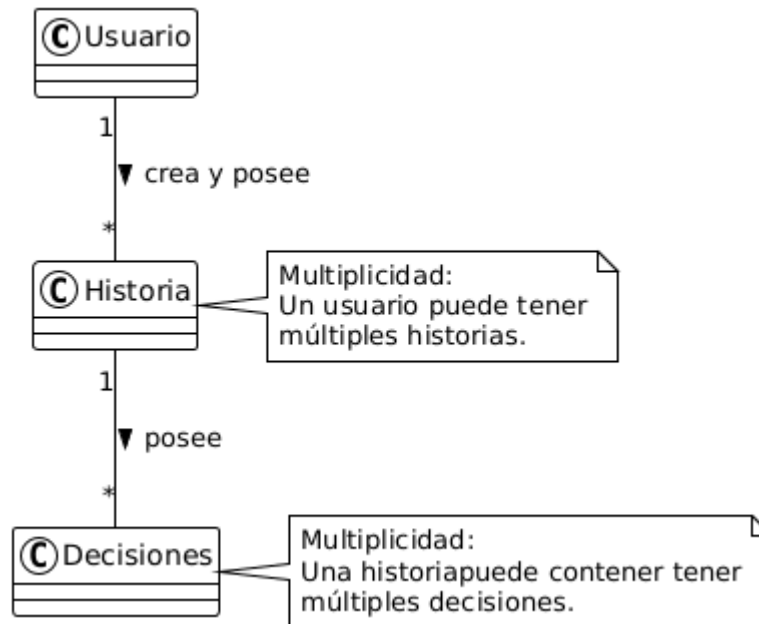
Este punto de vista se centra en representar el sistema desde la perspectiva de clases, objetos y sus relaciones, mostrando cómo se organiza la información y la funcionalidad a nivel conceptual. Describe la arquitectura de software de forma abstracta, mostrando qué entidades existen, qué responsabilidades tienen y cómo se relacionan entre sí.

## Preocupaciones que Aborda (Concerns):

Estructura de Datos: Definir cómo se modelan y almacenan conceptualmente los usuarios, historias, decisiones y progresos.

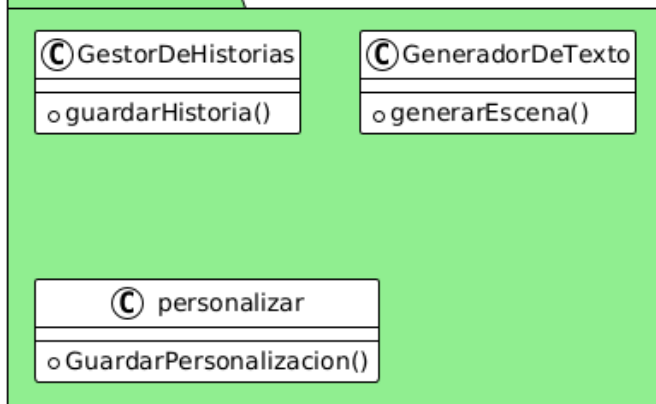


Consistencia de Relaciones: Evitar redundancias o inconsistencias en los vínculos entre las clases del dominio.

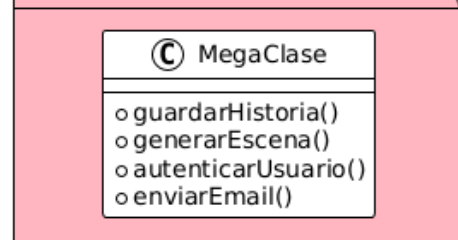


Separación de Responsabilidades: Asegurar que cada clase tenga un rol claro y no mezcle múltiples funciones.

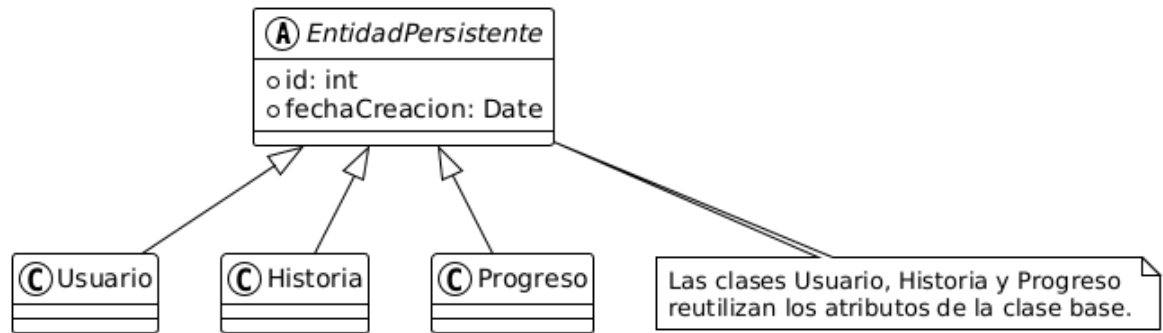
#### Diseño Correcto



#### Diseño Incorrecto (Anti-Patrón)



Reutilización: Diseñar clases que sean reutilizables en diferentes contextos de la aplicación.



Anti-preocupaciones: Este viewpoint no describe aspectos físicos de despliegue ni la dinámica de los procesos en el tiempo.

### Stakeholders Típicos:

Desarrolladores: Para tener un diseño lógico claro que puedan transformar en código.

Para entender las clases y sus relaciones al generar casos de prueba unitarios.

Para validar la corrección de las relaciones y la adecuación del modelo lógico a los requisitos.

### Tipos de Modelos (Model Kinds):

Diagrama de Clases UML: Para representar las clases del sistema, sus atributos, operaciones y asociaciones.

Diagrama de Objetos UML: Para mostrar instancias concretas de las clases y cómo interactúan en un escenario particular.

### Operaciones sobre las Vistas:

Creación: Se construye a partir de los requisitos funcionales y las entidades identificadas en el dominio narrativo.

Interpretación: Permite a los desarrolladores y arquitectos comprender cómo se organiza la lógica del sistema.

Análisis: Ayuda a verificar que no se omitió ninguna entidad clave y que las relaciones son coherentes.

Diseño/Implementación: Sirve como un modelo directo para codificar las clases y sus interacciones.

### **Fuentes:**

ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description.

Documentación del proyecto “Historia Interactiva IA”.

## **3.4. Interaction Viewpoint (Punto de Vista de Interacción)**

### **Nombre del Viewpoint:**

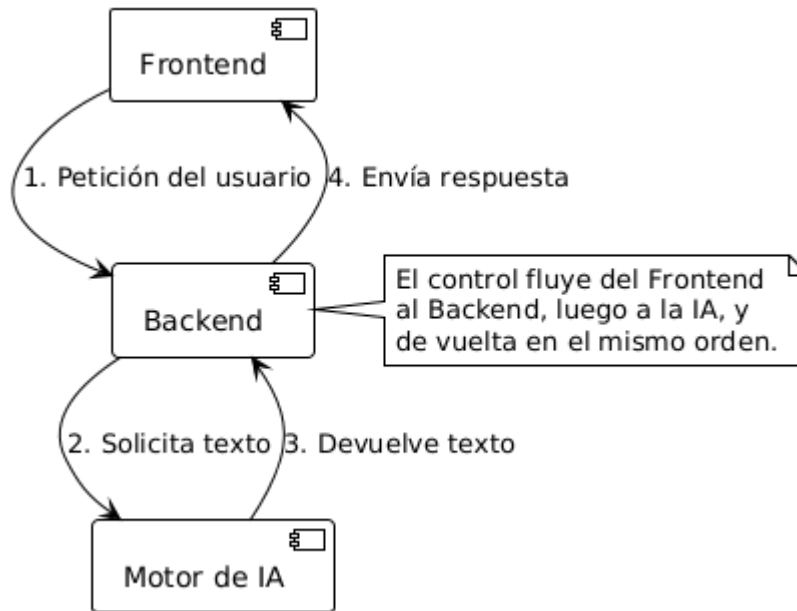
Interaction Viewpoint (Punto de Vista de Interacción). A veces también llamado Process Viewpoint o Behavioral Viewpoint.

### **Resumen:**

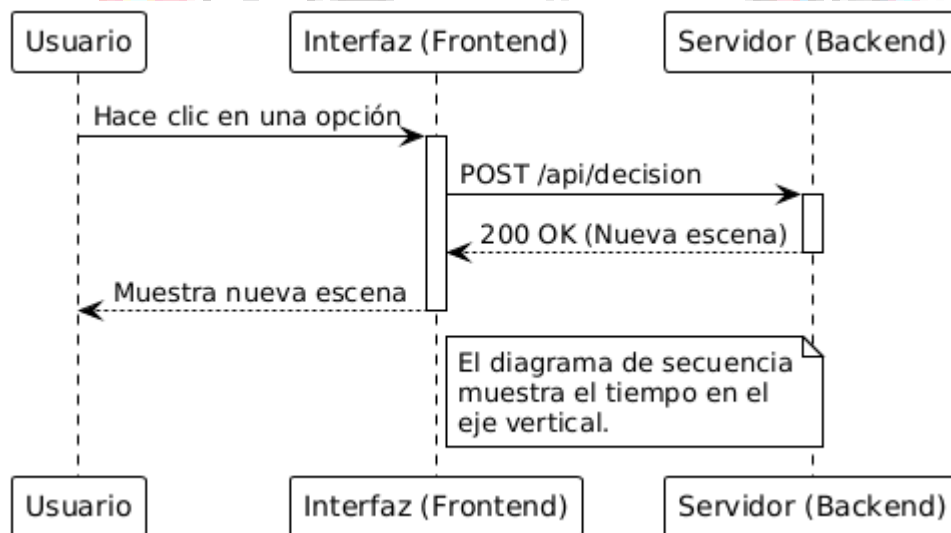
Este punto de vista se centra en el comportamiento dinámico del sistema. Describe cómo los componentes y clases interactúan entre sí a lo largo del tiempo para llevar a cabo una función o un escenario específico. Mientras que otros viewpoints muestran la estructura estática, este muestra el sistema "en acción".

### **Preocupaciones que Aborda (Concerns):**

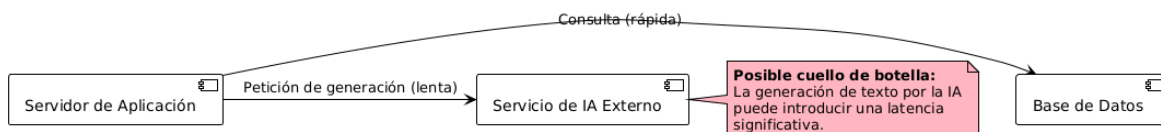
Flujo de Control: ¿Cómo se comunican los componentes en tiempo de ejecución?



Secuencia de Eventos: ¿En qué orden ocurren las operaciones para completar una tarea (ej. "generar un nuevo segmento de historia")?

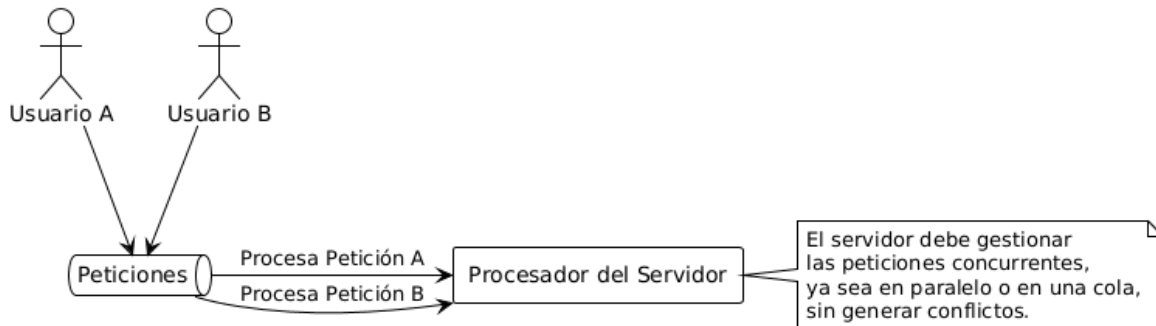


Rendimiento en Tiempo Real: ¿Dónde pueden ocurrir cuellos de botella o latencias en la comunicación entre componentes?





Manejo de Concurrencia: ¿Cómo maneja el sistema múltiples peticiones de usuarios simultáneamente?



### Stakeholders Típicos:

Desarrolladores: Para entender la lógica de ejecución y cómo se orquestan las llamadas entre servicios.

Administrador del sistema: Para analizar el comportamiento del sistema

### Tipos de Modelos (Model Kinds):

Diagrama de Secuencia UML: Ideal para mostrar la secuencia cronológica de mensajes entre objetos o componentes para un escenario específico.

Diagrama de Actividad UML: Útil para modelar el flujo de trabajo (workflow) de un proceso de alto nivel.

Diagrama de Comunicación UML: Para visualizar las interacciones entre objetos centrándose en los mensajes que intercambian.

### Operaciones sobre las Vistas:

Creación: Se modelan a partir de los casos de uso más críticos o complejos del sistema.

Análisis: Permite identificar dependencias en tiempo de ejecución, posibles puntos de fallo en la comunicación y optimizar los flujos de datos.

### Fuentes:

- ISO/IEC/IEEE 42010:2011.
- Kruchten, P.B., "The '4+1' View Model of Architecture".

### 3.5. Deployment Viewpoint (Punto de Vista de Despliegue)

#### Nombre del Viewpoint:

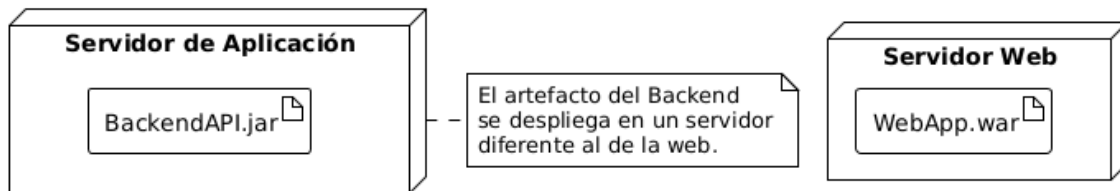
Deployment Viewpoint (Punto de Vista de Despliegue). También conocido como Physical Viewpoint.

#### Resumen:

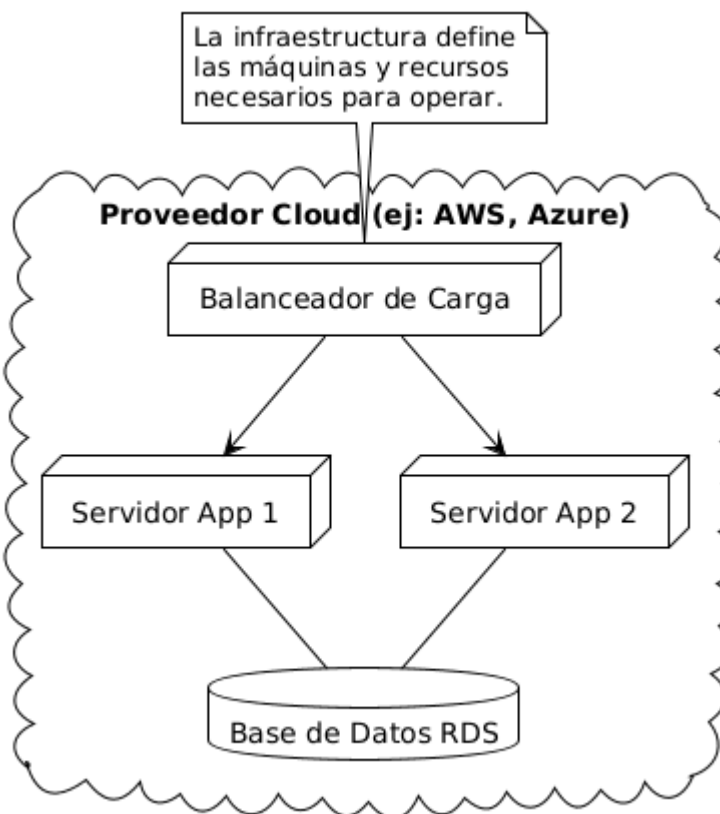
Este viewpoint describe la topología física del sistema, mostrando cómo los componentes de software se mapean a la infraestructura de hardware subyacente. Aborda el entorno de ejecución, los nodos físicos (servidores, bases de datos) y las conexiones de red entre ellos.

#### Preocupaciones que Aborda (Concerns):

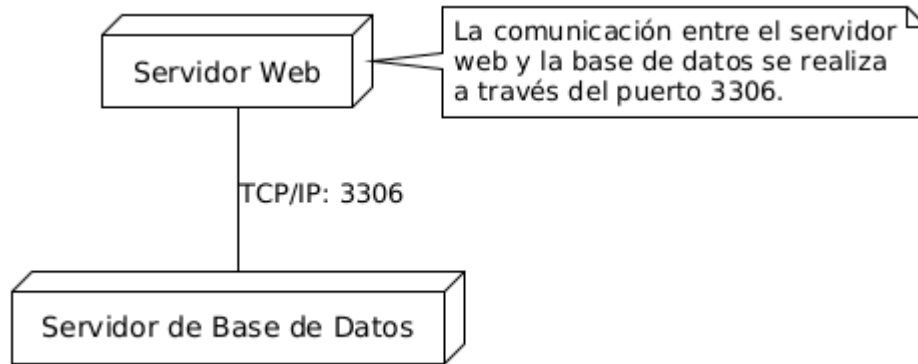
Distribución del Software: ¿Qué componente de software se ejecuta en qué servidor o contenedor?



Infraestructura de Hardware: ¿Qué servidores, balanceadores de carga, y bases de datos se necesitan?



Comunicación en Red: ¿Cómo se comunican los nodos físicos entre sí (protocolos, puertos)?



Escalabilidad y Disponibilidad: ¿Cómo se puede escalar la infraestructura para soportar más carga? ¿Cómo se asegura la redundancia para evitar puntos únicos de fallo?

### Stakeholders Típicos:

Administradores del Sistema Para planificar la provisión de infraestructura, el despliegue y el mantenimiento.

Desarrolladores: Para entender las limitaciones y características del entorno donde se ejecutará su código.

### Tipos de Modelos (Model Kinds):

Diagrama de Despliegue UML: Es el modelo estándar para representar la asignación de artefactos de software a nodos de hardware y las conexiones entre ellos.

### Operaciones sobre las Vistas:

**Creación:** Se diseña basándose en los requisitos no funcionales de rendimiento, escalabilidad y disponibilidad.

**Análisis:** Se utiliza para planificar la capacidad del sistema, analizar la seguridad de la red y diseñar estrategias de respaldo y recuperación.

### **Fuentes:**

ISO/IEC/IEEE 42010:2011.

## **3.6. Data Viewpoint (Punto de Vista de Datos)**

### **Nombre del Viewpoint:**

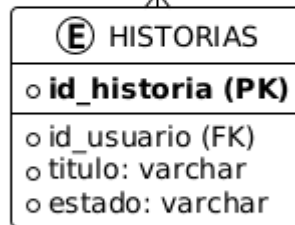
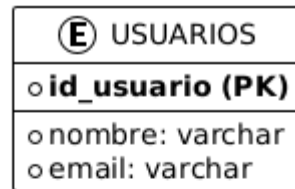
Data Viewpoint (Punto de Vista de Datos). A veces se solapa con el Information Viewpoint.

### **Resumen:**

Este viewpoint se enfoca exclusivamente en la estructura, almacenamiento y flujo de los datos a través del sistema. Mientras que el Logical Viewpoint describe las clases de la aplicación, este se centra en el esquema de la base de datos, la persistencia de la información y su ciclo de vida.

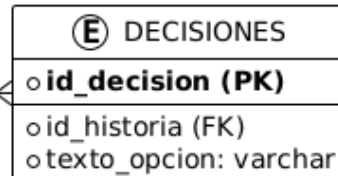
### **Preocupaciones que Aborda (Concerns):**

Esquema de la Base de Datos: ¿Cómo se estructuran las tablas (o colecciones en NoSQL) y sus relaciones?



Una historia pertenece a un único usuario.

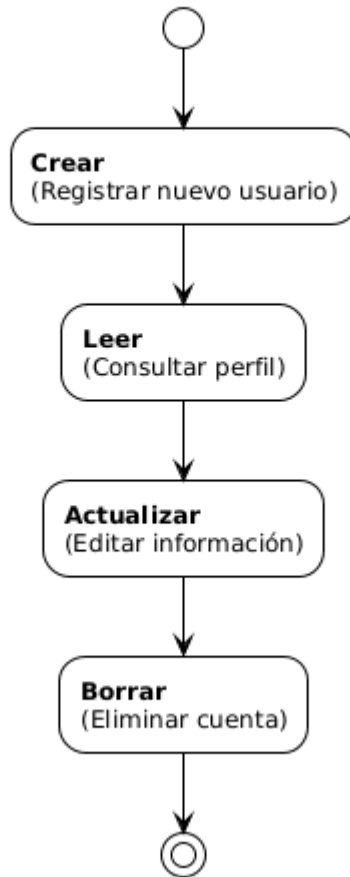
Integridad de Datos: ¿Qué restricciones se aplican para asegurar que los datos sean consistentes y válidos?



**Restricción (Foreign Key):**  
No puede existir una decisión si no está asociada a una historia válida.

Ciclo de Vida de los Datos: ¿Cómo se crean, leen, actualizan y eliminan los datos (CRUD)?

# Ciclo de Vida de un Usuario



Seguridad de los Datos: ¿Cómo se protegen los datos en reposo (cifrado)?



Los datos sensibles, como las contraseñas, no se guardan en texto plano. Se almacenan cifrados para protegerlos.

## Stakeholders Típicos:

Desarrolladores (Backend): Para implementar el acceso y la manipulación de los datos.

Administradores de Bases de Datos (DBA): Para diseñar, optimizar y mantener la base de datos.

## Tipos de Modelos (Model Kinds):

Diagrama de Entidad-Relación (ERD): El modelo clásico para diseñar la estructura de una base de datos relacional.

Modelos de Documentos (JSON/BSON): Para bases de datos NoSQL, mostrando la estructura de los documentos.

## Operaciones sobre las Vistas:

Creación: Se deriva del modelo lógico (Diagrama de Clases) y se refina para optimizar el almacenamiento y las consultas.

Análisis: Permite planificar estrategias de migración de datos, optimizar el rendimiento de las consultas y asegurar el cumplimiento de las normativas de protección de datos.

## Fuentes:

ISO/IEC/IEEE 42010:2011.

## 3.7. Architectural Patterns Viewpoint (Punto de Vista de Patrones Arquitectónicos)

### Nombre del Viewpoint:

Architectural Patterns Viewpoint (Punto de Vista de Patrones Arquitectónicos).

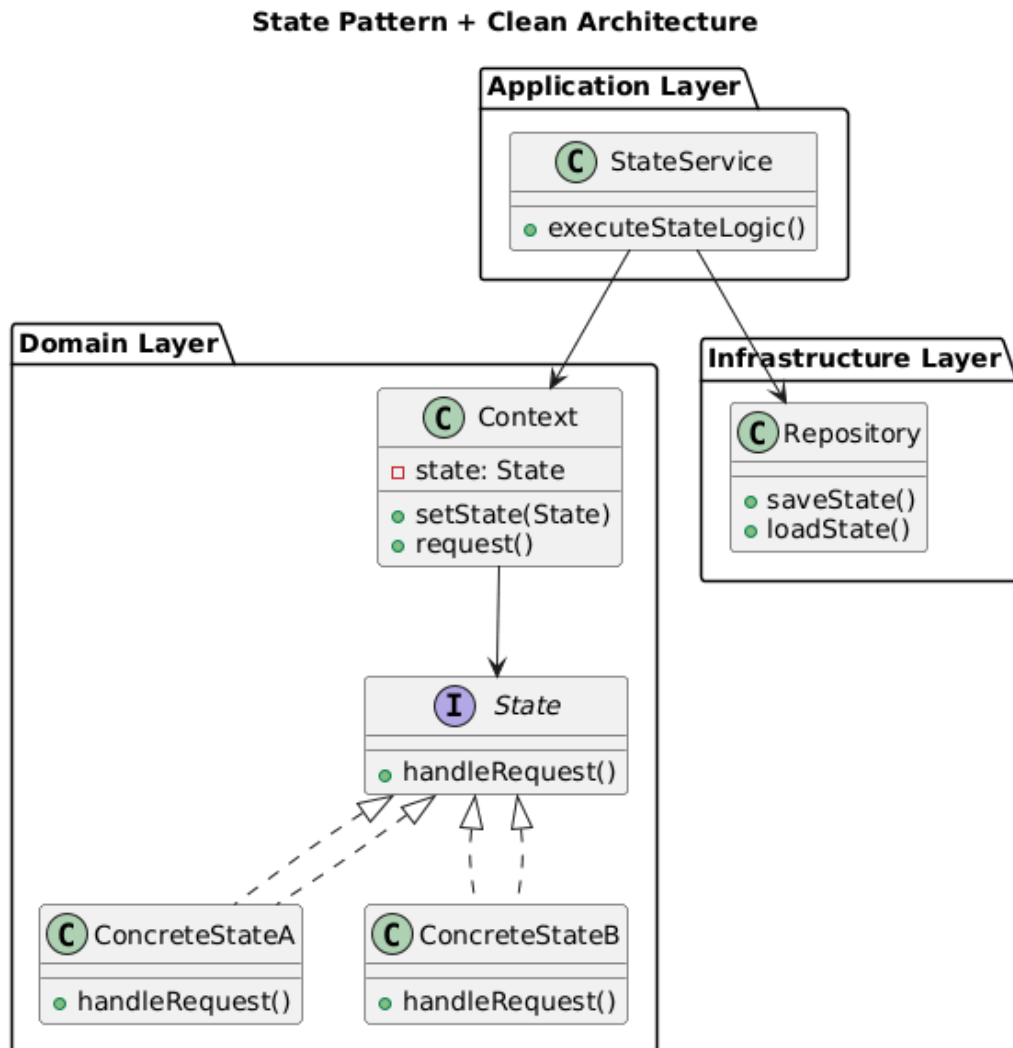
## Resumen:

Este punto de vista se enfoca en las estrategias de diseño fundamentales y reutilizables que conforman la arquitectura. No describe componentes individuales, sino las "recetas" o plantillas de alto nivel que se han utilizado para estructurar el sistema y resolver problemas comunes de diseño.



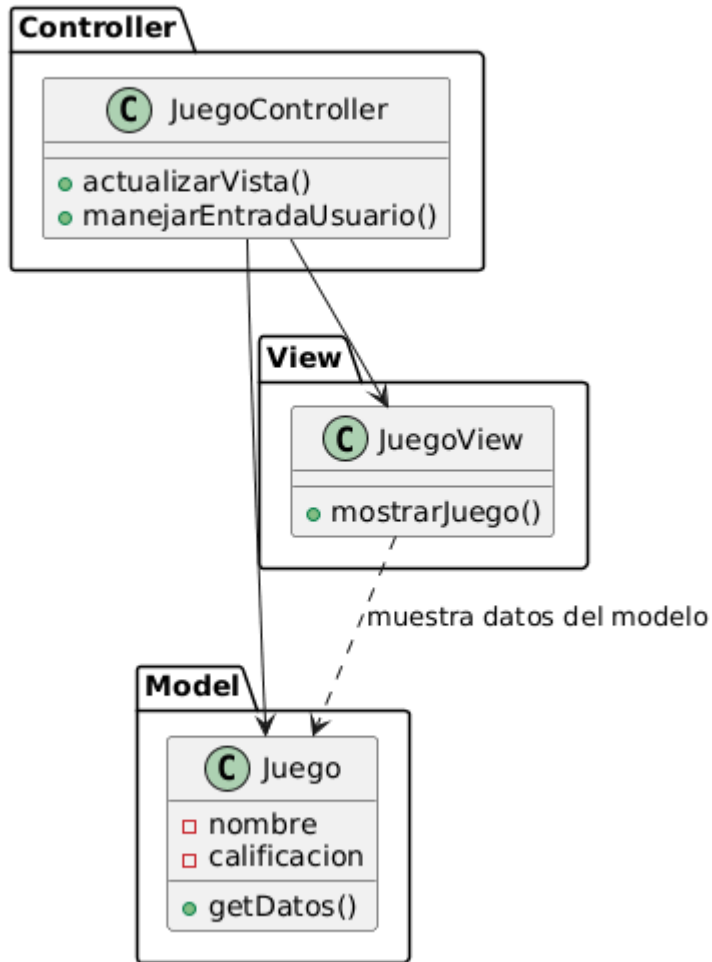
## Preocupaciones que Aborda (Concerns):

Estrategia de Diseño: ¿Cuáles son las decisiones de diseño más importantes que guían la estructura general?



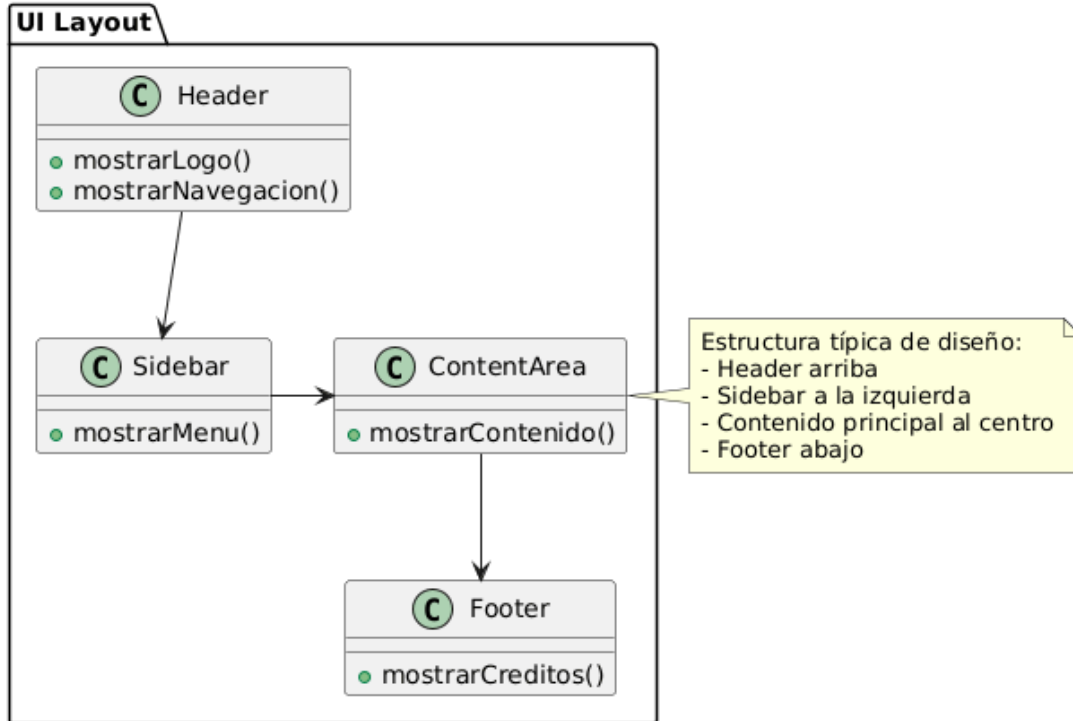
Calidad del Sistema: ¿Cómo se abordan atributos de calidad como la mantenibilidad, testeabilidad y separación de preocupaciones?

## Modelo - Vista - Controlador (MVC)



Consistencia: ¿Cómo se asegura que los componentes se desarrollen de una manera consistente y predecible?

## UI Layout Pattern (Estructura de Interfaz)



### Stakeholders Típicos:

Desarrolladores (especialmente los nuevos en el proyecto): Para entender rápidamente los principios de diseño y cómo deben construir nuevas funcionalidades.

### Tipos de Modelos (Model Kinds):

Diagrama Conceptual de Patrones: Un diagrama de alto nivel que muestra las principales áreas del sistema y qué patrón arquitectónico las gobierna.

## 4. Vistas de la Arquitectura

Esta sección contiene las vistas arquitectónicas del sistema "Historia Interactiva IA". Cada vista se rige por un viewpoint definido en la Sección 3 y presenta un

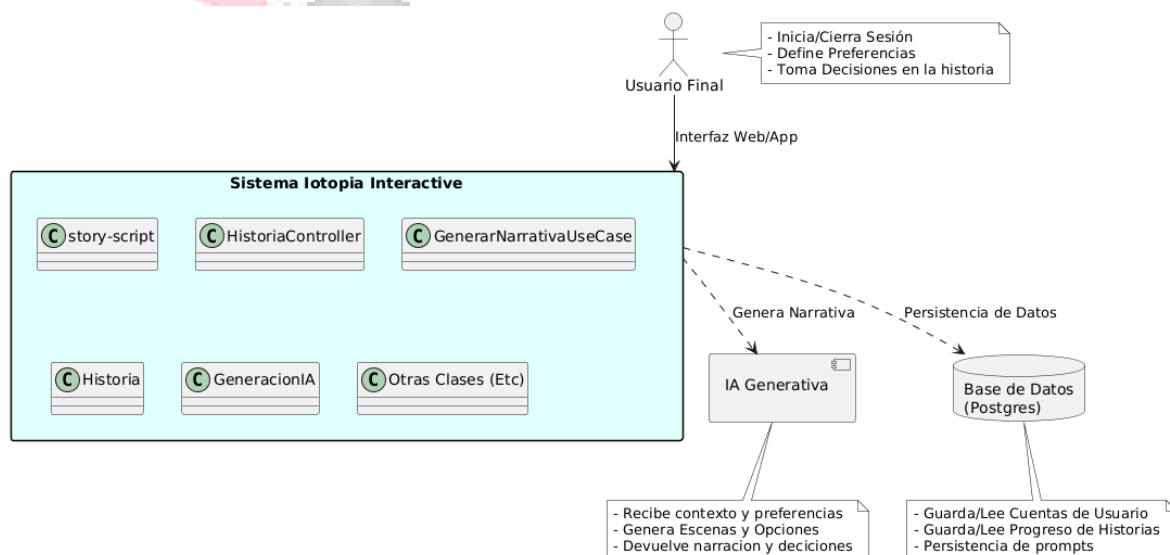
conjunto de modelos (diagramas y tablas) que describen el sistema desde una perspectiva particular.

## 4.1. Vista de Contexto del Sistema "Historia Interactiva IA (IotopiaIA)"

Gobernado por: Context Viewpoint (definido en la Sección 3.1).

Propósito: Esta vista delimita el alcance del sistema "Iotopia Interactive", estableciendo sus fronteras con respecto al entorno operativo. Su objetivo principal es identificar las interacciones entre el sistema y las entidades externas (actores humanos y sistemas de soporte), definiendo claramente qué es responsabilidad del desarrollo interno y qué servicios son consumidos como recursos externos.

Modelo: Diagrama de Contexto El siguiente modelo representa al sistema como una entidad central que orquesta la interacción entre el usuario final y los servicios de infraestructura crítica (IA y Persistencia).



| Actor Externo            | Flujo de Información<br>(Sistema -> Actor)                                                      | Flujo de Información<br>(Actor -> Sistema)                                           | Descripción de<br>la Interacción                                                                |
|--------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Usuario Final            | - Escenas de la historia - Opciones de decisión - Mensajes de estado (progreso guardado, error) | - Credenciales (email/contraseña) - Preferencias de la historia - Elección de opción | El usuario interactúa con el sistema para vivir la experiencia narrativa y gestionar su cuenta. |
| Sistema de Autenticación | - Petición de validación de credenciales                                                        | - Respuesta de éxito/fallo de autenticación                                          | El sistema delega la validación de la identidad del usuario a este servicio externo.            |
| Motor de IA Narrativa    | - Preferencias de la historia - Decisión tomada por el usuario - Estado actual de la historia   | - Texto de la nueva escena - Conjunto de 3+ opciones - Ajuste de dificultad          | El sistema utiliza este servicio para la generación dinámica de todo el contenido narrativo.    |
| Base de Datos            | - Escritura (INSERT/UPDATE) de datos de usuario e historias. - Peticiones de lectura (SELECT).  | - Datos de usuario solicitados. - Estado de la historia guardada.                    | El sistema utiliza la base de datos para almacenar toda la información persistente.             |

## Explicación del Diagrama y Elementos

Este diagrama ilustra el entorno de operación del sistema:

1. Sistema Itopia Interactive (Límite del Sistema): Representa el núcleo de la solución de software. Contiene toda la lógica de negocio, controladores (MVC) y casos de uso (Clean Architecture). Es la "caja blanca" donde residen los componentes desarrollados específicamente para el proyecto (como HistoriaController, GenerarNarrativaUseCase, etc.). Su

responsabilidad es orquestar las peticiones del usuario y transformarlas en instrucciones para la IA o consultas a la base de datos.

2. Usuario Final (Actor Principal): Es el iniciador de todas las interacciones. Su rol es activo: define las preferencias (género, ambientación), toma decisiones clave y consume la narrativa. El sistema depende completamente de los inputs de este actor para avanzar en la máquina de estados de la historia.

3. IA Generativa (Sistema Externo / Servicio): Representa el motor de inteligencia artificial (ej. API de OpenAI). Se modela como un actor externo porque el sistema no controla la lógica interna de la IA, sino que la consume.

Interacción: Recibe "prompts" estructurados con el contexto y devuelve texto narrativo y opciones.

4. Base de Datos Postgres (Sistema Externo / Infraestructura): Aunque es parte del stack tecnológico, arquitectónicamente se considera un actor externo (Infraestructura). El sistema envía datos para su persistencia y solicita recuperación de estado, pero la gestión física de los datos y el motor de base de datos operan independientemente de la lógica de la aplicación.

#### Justificación Arquitectónica y Resolución de Preocupaciones:

A continuación se detalla por qué el sistema se estructuró de esta manera y cómo responde a las preocupaciones de los interesados (stakeholders):

1. ¿Por qué la Base de Datos y la IA son actores externos? (Principio de Clean Architecture) Esta decisión de diseño responde a la preocupación de Mantenibilidad y Desacoplamiento. Siguiendo los principios de Clean Architecture:-

- La IA es un detalle: Si en el futuro se decide cambiar de GPT a Llama o DeepSeek, el sistema central no debería verse afectado estructuralmente. Al modelar la IA como un actor externo conectado mediante interfaces, protegemos la lógica de negocio (Entidades y Casos de Uso) de los cambios en proveedores externos.
- La Base de Datos es un detalle: El sistema interactúa con la BD a través de repositorios. Al situarla fuera del límite del sistema en este diagrama, enfatizamos que la lógica de la historia (la narrativa, los estados) es independiente de si los datos se guardan en PostgreSQL, MySQL o un archivo de texto.

2. Resolución de la Preocupación de Interoperabilidad Los desarrolladores y arquitectos necesitan saber con qué debe comunicarse el sistema. Este diagrama resuelve esta preocupación al explicitar que el sistema requiere conexión a internet para la API de IA y acceso a un servidor de base de datos. Define los puntos de integración críticos sin entrar en la complejidad del código interno.

3. Aplicación del Patrón MVC en el Contexto Aunque es una vista de alto nivel, se observa la influencia del patrón MVC en la frontera con el Usuario:

- El Usuario interactúa a través de la "Interfaz Web" (Vistas).
- El Sistema recibe esas interacciones mediante "Controladores" (HistoriaController). Esto justifica por qué el usuario no tiene acceso directo a la base de datos ni a la IA; todo debe pasar por el flujo controlado del sistema.

4. Consistencia de Datos y Estado (State Pattern) El diagrama muestra el flujo circular necesario para el patrón de estados (State Pattern):

1. El sistema lee el estado actual de la Base de Datos.
2. Envía el contexto a la IA.
3. Presenta el resultado al Usuario. Esta separación asegura que la IA no necesita "recordar" el estado por sí misma (ya que las APIs suelen ser stateless), sino que es el Sistema quien inyecta el contexto necesario en cada interacción, garantizando la coherencia narrativa.

## 4.2. Vista de Composición del Sistema ( Estructura)

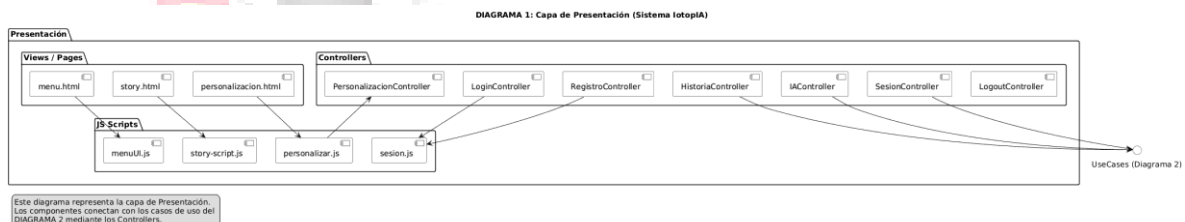
Gobernado por: Composition/Structural Viewpoint.

Propósito: Esta vista detalla la estructura interna del sistema "Ilotopia Interactive", descomponiéndolo en subsistemas y componentes de software discretos. Dado que el sistema sigue una Arquitectura Limpia (Clean Architecture) y el patrón MVC, se ha optado por representar esta vista a través de tres diagramas complementarios que corresponden a las capas arquitectónicas. Esto permite visualizar con claridad la separación de responsabilidades, el flujo de dependencia y la encapsulación de la lógica de negocio.

A continuación, se presentan los modelos de composición por capas:

### 4.2.1. Capa de Presentación (Interfaz y Controladores)

El primer nivel de descomposición aborda cómo el sistema interactúa con el usuario y procesa las solicitudes iniciales.



Explicación y Justificación Arquitectónica: Este diagrama ilustra la implementación del patrón Model-View-Controller (MVC) en la frontera del sistema.

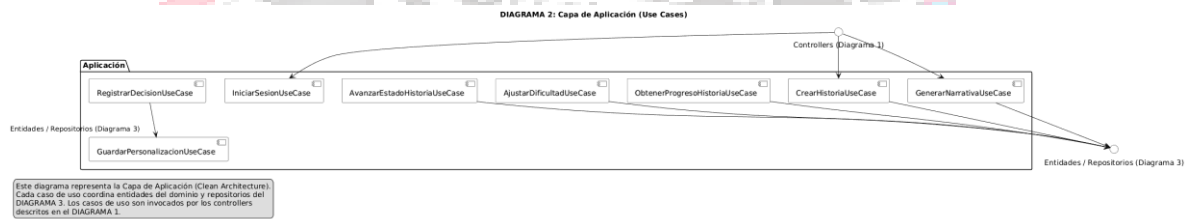
- Separación de Vistas y Lógica: Las "Views" (.html) y "JS Scripts" están completamente desacoplados de la lógica de negocio profunda. Esto resuelve la preocupación de modificabilidad: se puede rediseñar la interfaz gráfica (story.html) o cambiar los scripts de animación (story-script.js) sin riesgo de romper las reglas de la historia o la conexión con la base de datos.



- Rol de los Controladores: Los controladores (IAController, HistoriaController) actúan como "Gateway" o puerta de entrada. Su única responsabilidad es recibir la petición HTTP, validarla y delegar el trabajo a la capa de Aplicación (Diagrama 2). Esto mantiene los controladores "delgados" y facilita las pruebas unitarias.

#### 4.2.2. Capa de Aplicación (Casos de Uso)

El segundo nivel representa el núcleo funcional del sistema, donde reside la orquestación de la lógica.



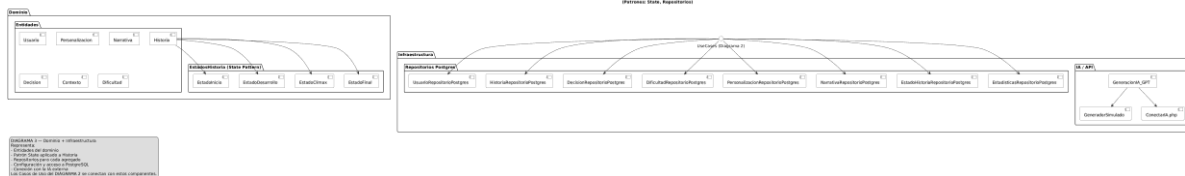
Explicación y Justificación Arquitectónica: Este diagrama representa la capa de "Casos de Uso" de la Clean Architecture.

- Independencia de Frameworks: Los componentes aquí listados (ej. GenerarNarrativaUseCase, AvanzarEstadoHistoriaUseCase) contienen reglas de aplicación puras. No saben si la petición vino de una web o una app móvil, ni saben si la base de datos es SQL o NoSQL. Solo orquestan el flujo.
- Resolución de la Complejidad del Negocio: Al encapsular cada acción del usuario en una clase específica (un Caso de Uso), el sistema resuelve la preocupación de mantenibilidad. Si hay un error en cómo se genera la narrativa, el desarrollador sabe exactamente que debe revisar

GenerarNarrativaUseCase sin tener que buscar en miles de líneas de código mezclado.

#### 4.2.3. Capa de Dominio e Infraestructura (Persistencia y Estado)

El tercer nivel detalla las entidades fundamentales, el manejo de estados y la conexión con servicios externos.



Explicación y Justificación Arquitectónica: Este diagrama es crítico porque muestra cómo se resuelven dos de los desafíos más grandes del proyecto: la gestión del flujo narrativo y la persistencia de datos.

##### 1. Patrón State (Gestión de la Narrativa):

- Por qué se usa: La narrativa interactiva tiene fases estrictas (Inicio, Desarrollo, Clímax, Final). Usar simples condicionales (if/else) haría el código inmanejable.
- Implementación: El paquete EstadosHistoria muestra la aplicación del State Pattern. La clase Historia delega el comportamiento al estado actual (ej. EstadoInicio). Esto permite que cada estado defina sus propias reglas de transición y dificultad, garantizando que el sistema sea robusto frente a inconsistencias lógicas (ej. no saltar del Inicio al Final sin pasar por el Desarrollo).

## 2. Patrón Repository (Infraestructura de Datos):

- Por qué se usa: Para desacoplar el dominio de la base de datos física.
- Implementación: Los Casos de Uso (del Diagrama 2) no hacen consultas SQL directas. En su lugar, piden datos a las interfaces de los repositorios (HistoriaRepositorioPostgres). Esto permite cambiar el motor de base de datos en el futuro sin tocar la lógica de negocio, resolviendo la preocupación de escalabilidad y evolución tecnológica.

## 3. Abstracción de la IA:

- La infraestructura muestra GeneracionIA\_GPT y GeneradorSimulado. Esto demuestra un diseño preparado para fallos o pruebas. Si la API de GPT cae, el sistema puede cambiar a GeneradorSimulado (polimorfismo), asegurando la disponibilidad del servicio.

La división en estos tres diagramas demuestra que el sistema "Itopia Interactive" no es un sistema desordenado, sino una estructura modular donde:

1. La Presentación solo se preocupa por mostrar datos.
2. La Aplicación coordina qué se debe hacer.
3. El Dominio define las reglas y el estado.
4. La Infraestructura provee los recursos técnicos (BD, IA).

## 4.3. Vista Lógica del Sistema "Historia Interactiva IA"

- Gobernado por: Logical Viewpoint (definido en la Sección 3.3).

Propósito: Esta vista se centra en la realización funcional de los requisitos del sistema. Su objetivo es describir cómo el sistema está estructurado en términos de clases, paquetes y capas lógicas, mostrando las abstracciones clave y los mecanismos de colaboración. A diferencia de las vistas anteriores que mostraban componentes físicos o fronteras, esta vista detalla la organización interna del código fuente y cómo este implementa los patrones arquitectónicos seleccionados.

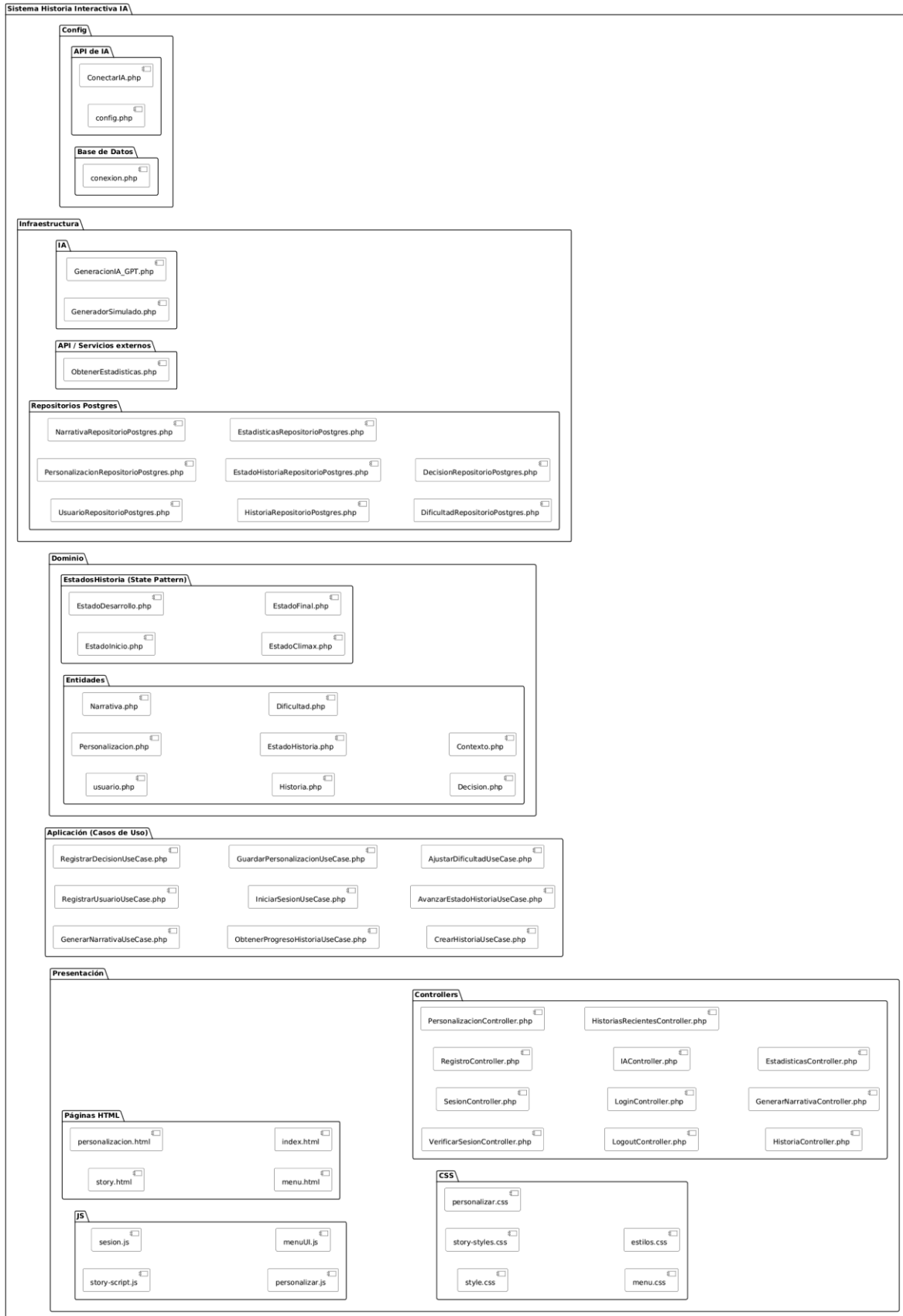
- A continuación se presentan dos niveles de detalle: la organización general de paquetes y el diagrama de clases detallado de los componentes críticos.

#### 4.3.1. Organización Estructural por Capas (Paquetes)

- Este modelo presenta la estructura del código fuente y responsabilidades.

Aclaracion se detallan las clases mas importantes pues el diagrama quedaria grandisimo y no se podría apreciar





## ustificación Arquitectónica y Resolución de Preocupaciones:

### 1. Separación de la Capa de Presentación (MVC):

- Por qué: Se observa una división clara entre JS/HTML (Vistas) y Controllers (Controladores PHP). Esta estructura resuelve la preocupación de Desacoplamiento UI/Lógica. Permite que los diseñadores frontend trabajen en los archivos HTML/CSS sin riesgo de alterar la lógica de negocio en PHP, y viceversa.

### 2. Aislamiento del Dominio:

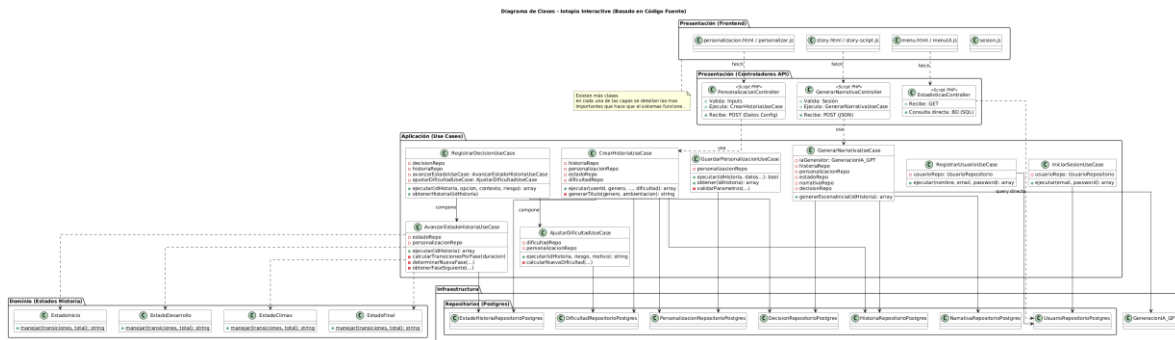
- Por qué: El paquete Dominio no tiene dependencias hacia afuera. Contiene las Entidades (Historia, Narrativa) y las reglas de negocio puras (EstadosHistoria). Esto asegura que las reglas fundamentales del "juego" sean independientes de la tecnología. Si mañana cambiamos PostgreSQL por MongoDB, el Dominio permanece intacto.

### 3. Capa de Aplicación como Orquestador:

- Por qué: La capa Aplicación (Casos de Uso) actúa como intermediaria. Contiene la lógica específica de la aplicación (ej. GenerarNarrativaUseCase). Esto centraliza las operaciones del sistema, facilitando la auditoría y el mantenimiento.

## 4.3.2. Diseño Detallado de Clases y Relaciones

Este diagrama profundiza en las clases más críticas del sistema, exponiendo sus métodos principales y cómo colaboran para cumplir con los requisitos funcionales complejos, como la generación de narrativa y el control de estados.



## Explicación del Diseño y Patrones Aplicados:

### 1. Implementación de Casos de Uso (Clean Architecture):

- Diseño: Clases como `RegistrarDecisionUseCase` o `GenerarNarrativaUseCase` encapsulan una única responsabilidad ("Single Responsibility Principle").
- Justificación: Observamos que `RegistrarDecisionUseCase` compone otros casos de uso (`AvanzarEstado`, `AjustarDificultad`). Esto demuestra un diseño modular donde se reutiliza lógica existente en lugar de duplicarla. Esto resuelve la preocupación de Mantenibilidad y Reusabilidad.

### 2. Patrón State (Gestión de Fases):

- Diseño: El paquete de Dominio incluye clases específicas (`EstadoInicio`, `EstadoDesarrollo`, etc.) que son utilizadas por `AvanzarEstadoHistoriaUseCase`.
- Por qué: En lugar de tener un gigantesco switch o if-else anidado dentro de una clase `Historia` para saber qué pasa después, cada clase de Estado encapsula sus propias reglas de transición.
- Beneficio: Si se desea agregar una fase nueva (ej. "Epílogo") o cambiar las reglas del "Clímax", solo se modifica una clase pequeña sin riesgo de romper el resto del flujo narrativo. Esto responde a la preocupación de Extensibilidad.

### 3. Patrón Repository e Inversión de Dependencias:

- Diseño: Los Casos de Uso tienen atributos como `- historiaRepo` o `- decisionRepo`.

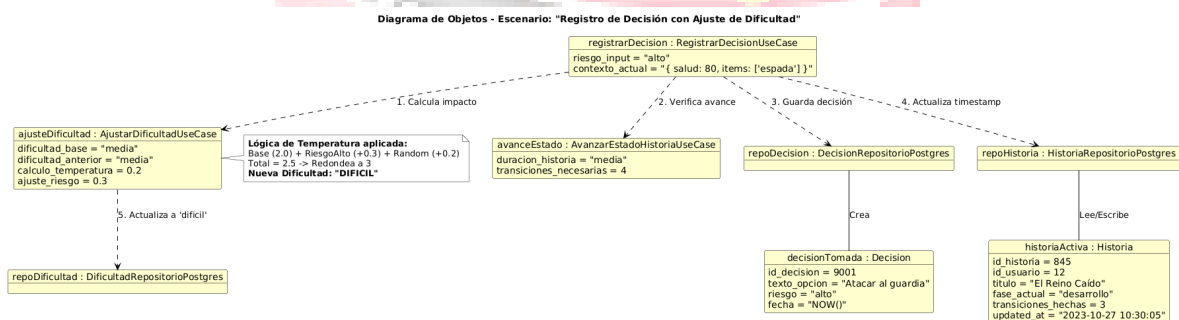
- Por qué: El diagrama muestra que los Casos de Uso dependen de abstracciones (implícitas en el diseño lógico) y no de la conexión directa a la base de datos. Las implementaciones concretas (UsuarioRepositorioPostgres) residen en la capa de Infraestructura.
- Beneficio: Permite realizar pruebas unitarias de la lógica de negocio simulando (mocking) la base de datos, lo cual es vital para asegurar la Calidad del Software.

#### 4. Controladores "Sencillos" :

- Diseño: Clases como GenerarNarrativaController tienen muy poca lógica; solo reciben POST, validan sesión y llaman a ejecutar() del Caso de Uso.
- Por qué: Se evita la "Lógica de Negocio en el Controlador". Esto asegura que la lógica narrativa sea accesible desde diferentes puntos de entrada (API REST, Web, CLI) sin duplicar código.

#### 4.3.3. Modelo Dinámico (Diagrama de Objetos)

Mientras que los diagramas anteriores son estáticos, este modelo captura un "instante en el tiempo" (snapshot) durante la ejecución de una funcionalidad crítica: el registro de una decisión del usuario.



Análisis del Escenario: Registro de Decisión y Ajuste de Dificultad:

Este diagrama valida que la arquitectura propuesta funciona dinámicamente para resolver requisitos complejos.

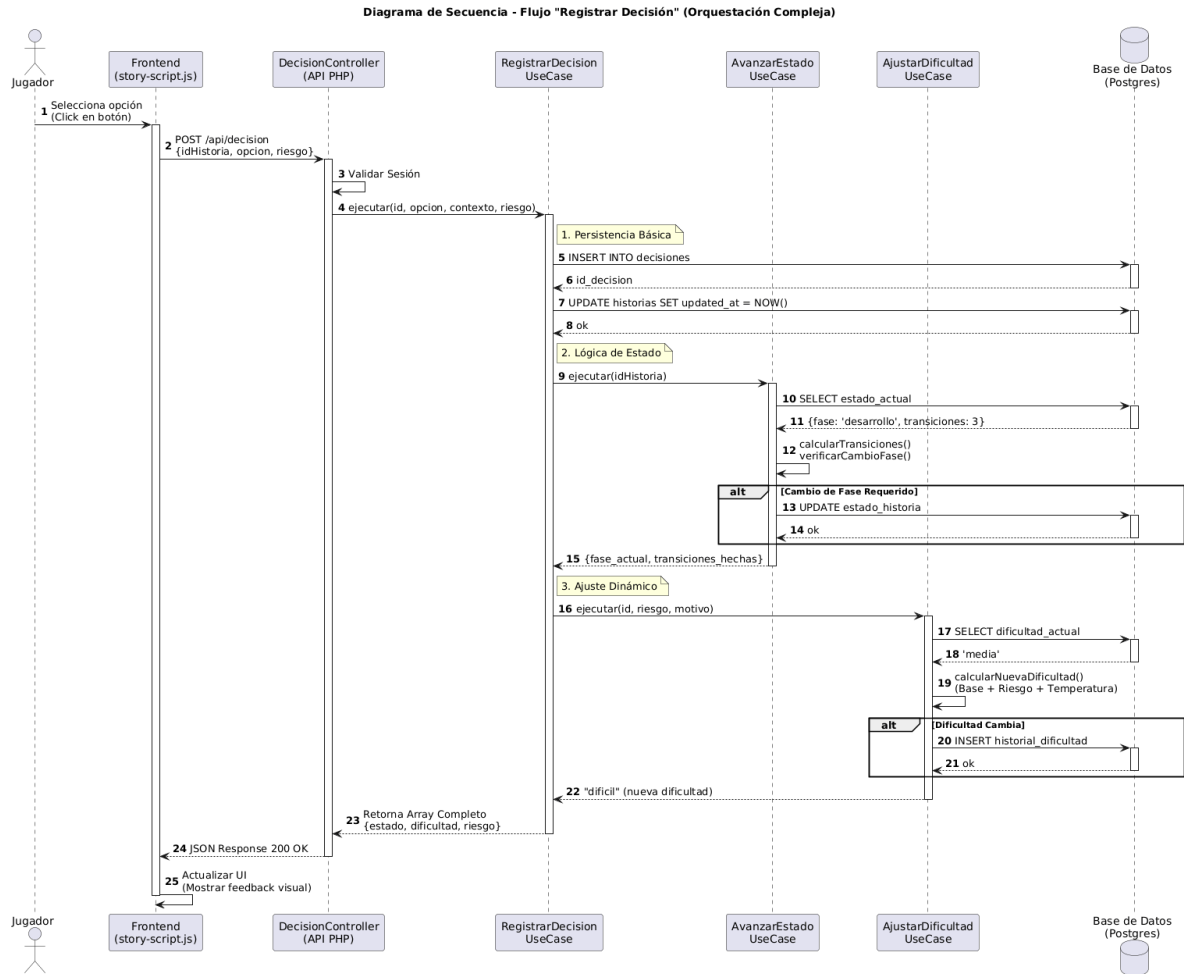


1. Flujo de Datos en Tiempo Real: El objeto UC\_Main (RegistrarDecision) actúa como orquestador. Recibe un input ("riesgo alto") y coordina a los objetos auxiliares (UC\_Diff, UC\_State) para procesar las consecuencias de esa decisión.
2. Cálculo de Temperatura (Algoritmo de Dificultad): La nota en el diagrama explica cómo funciona la lógica interna: la dificultad no es estática. El objeto UC\_Diff toma la dificultad base, suma el riesgo de la decisión actual y un factor aleatorio (temperatura) para determinar el nuevo estado (DIFICIL).
  - Por qué es importante: Demuestra que el sistema es adaptativo, cumpliendo con la promesa de una "IA Interactiva y Dinámica".
3. Persistencia Transaccional: Se observa que múltiples repositorios (RepoDecision, RepoHistoria, RepoDiff) son invocados en una misma operación lógica. Esto sugiere que, aunque las clases están separadas, trabajan en conjunto para asegurar la Integridad de los Datos (todo se guarda o nada se guarda).

#### 4.4. Vista de Interacción del Sistema "Historia Interactiva IA"

- **Gobernado por: Interaction Viewpoint (definido en la Sección 3.4).**
- **Propósito:** Esta vista describe el comportamiento dinámico del sistema. A diferencia de las vistas anteriores que eran estáticas (mostrando "qué es"), esta vista muestra "cómo funciona" a través de escenarios de uso específicos.

**Modelo: Diagrama de Secuencia** - Flujo "Registrar Decisión" El siguiente modelo detalla el escenario más crítico del sistema: el momento en que el usuario toma una decisión, desencadenando una serie de procesos internos para actualizar la narrativa, el estado y la dificultad.



## Explicación del Flujo y Justificación Arquitectónica

El diagrama ilustra una operación transaccional compleja dividida en pasos lógicos. A continuación se justifica por qué el sistema funciona de esta manera y cómo intervienen los patrones arquitectónicos:

### 1. El Rol del Controlador como "Gateway" (Patrón MVC)

- **Funcionamiento:** La interacción comienza con el DecisionController. Observamos que este componente no realiza ninguna lógica de negocio. Solo valida la sesión y delega inmediatamente la ejecución al caso de uso principal (MainUC).
- **Por qué se hizo así:** Esto responde a la preocupación de Desacoplamiento. Al mantener el controlador "delgado" (thin controller), aislamos la lógica del

juego de la capa HTTP. Si mañana decidimos cambiar la API por Websockets o una interfaz de línea de comandos, la lógica interna (MainUC) no necesita ser modificada.

## 2. Orquestación mediante Composición de Casos de Uso (Clean Architecture)

- **Funcionamiento:** El RegistrarDecisionUseCase (MainUC) actúa como un director de orquesta. No realiza todos los cálculos por sí mismo; en su lugar, llama a StateUC (Paso 2) y DiffUC (Paso 3).
- **Por qué resuelve preocupaciones de mantenimiento:** Esto aplica el Principio de Responsabilidad Única (SRP).
  - Si hay un error en cómo se calcula la dificultad, el desarrollador solo debe revisar DiffUC.
  - Si hay un error en el cambio de fase de la historia, se revisa StateUC.
  - Esto reduce drásticamente la complejidad ciclomática y facilita las pruebas unitarias aisladas.

## 3. Implementación del Patrón State en Tiempo de Ejecución

- **Funcionamiento:** En el Paso 2, el sistema consulta la base de datos para obtener el estado actual antes de decidir si avanzar.
- **Por qué funciona de esta manera:** La narrativa interactiva requiere coherencia estricta. El sistema no confía ciegamente en el frontend; verifica en el servidor (verificarCambioFase()) si se han cumplido las condiciones (número de transiciones) para pasar de "Desarrollo" a "Clímax". Esto garantiza la integridad del flujo narrativo incluso si el usuario intenta manipular la petición.

## 4. Adaptabilidad Dinámica (Lógica de Temperatura)

- **Funcionamiento:** En el Paso 3, el diagrama muestra cómo el sistema ajusta la dificultad basándose en el riesgo de la decisión actual y el historial previo.
- **Resolución de preocupaciones del Stakeholder (Game Design):** Una de las preocupaciones principales es que el juego "no sea aburrido". Al inyectar una lógica de ajuste dinámico (calcularNuevaDificultad) en cada interacción, el sistema asegura que la experiencia se personalice en tiempo real, respondiendo al comportamiento del usuario.

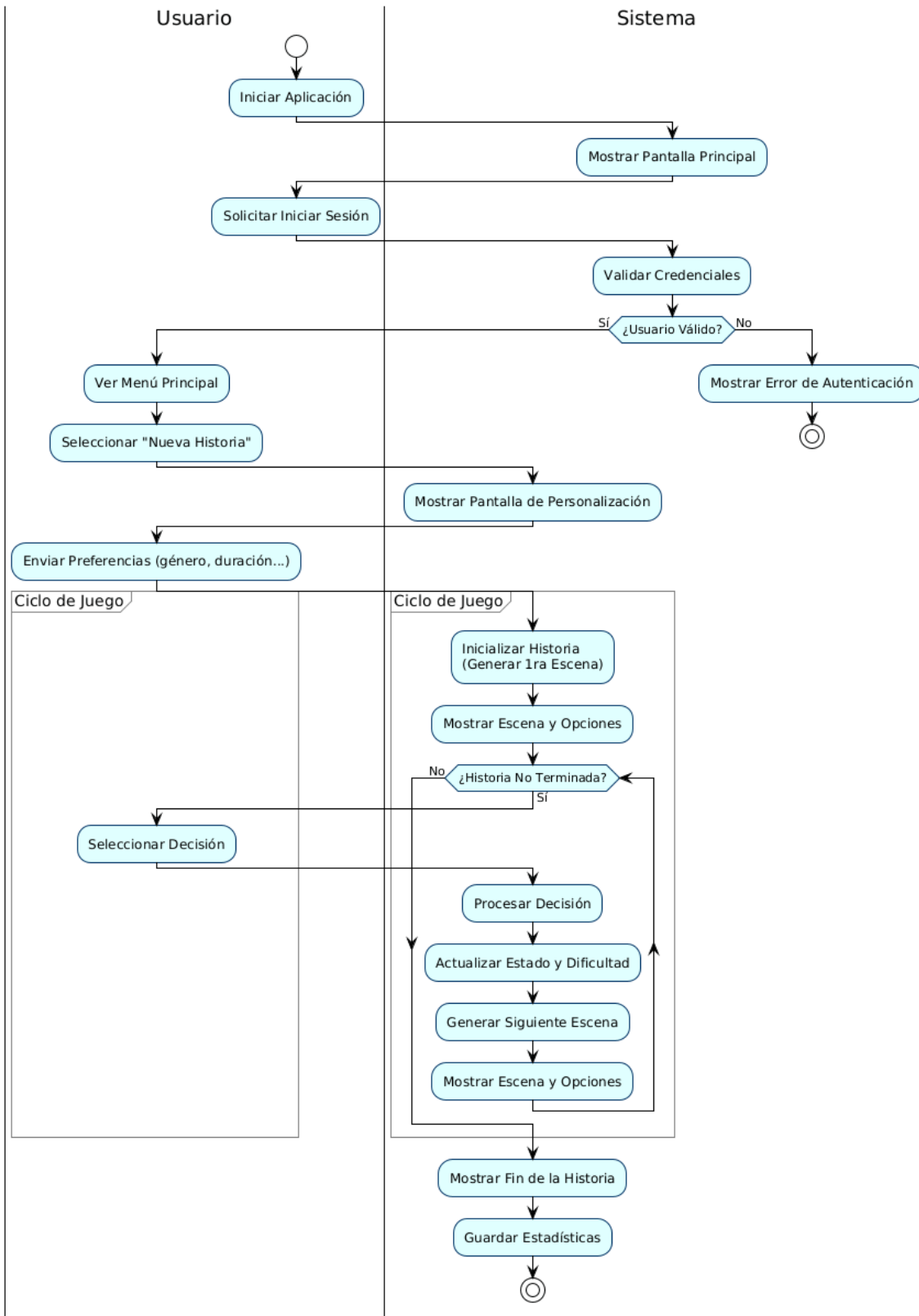
## 5. Persistencia y Eficiencia

- **Funcionamiento:** El diagrama muestra interacciones puntuales con la Base de Datos (INSERT, UPDATE, SELECT).
- **Por qué:** El sistema es stateless (sin estado en memoria entre peticiones PHP). Cada vez que el usuario actúa, el sistema debe reconstruir el contexto desde la BD, procesar y guardar. Esta arquitectura asegura que el servidor sea escalable, ya que no retiene memoria innecesaria entre los clics del usuario.

## Modelo 2: Diagrama de Actividad - Flujo de Trabajo del Usuario

Este diagrama modela el flujo de trabajo de alto nivel de un usuario dentro del sistema, desde que inicia hasta que termina una historia. Muestra los diferentes caminos y bucles que puede tomar la interacción.

## Flujo de Trabajo (Workflow) del Usuario



## Explicación del Diagrama de Actividad

Este diagrama divide el proceso en dos "carriles" (swimlanes): Usuario y Sistema, mostrando quién es responsable de cada acción.

1. Inicio y Autenticación: El flujo comienza con el Usuario iniciando la app y solicitando el login. El Sistema valida sus credenciales.
2. Decisión de Flujo (Bifurcación): El primer diamante (if) muestra una bifurcación. Si el usuario no es válido, el flujo termina en un error. Si es válido, continúa.
3. Personalización: El Usuario selecciona "Nueva Historia" y envía sus preferencias. El Sistema recibe estas preferencias.
4. Ciclo de Juego (Bucle):
  - El Sistema entra en la partición "Ciclo de Juego". Primero, inicializa la historia y muestra la escena inicial.
  - Comienza un bucle (while). Mientras la historia no esté en estado FINAL, el ciclo se repite.
  - El Usuario selecciona una decisión.
  - El Sistema recibe la decisión, la procesa, actualiza el estado (GestorDeEstados) y la dificultad (AdaptarDificultad), genera la siguiente escena (GeneracionIA) y se la vuelve a mostrar al Usuario.

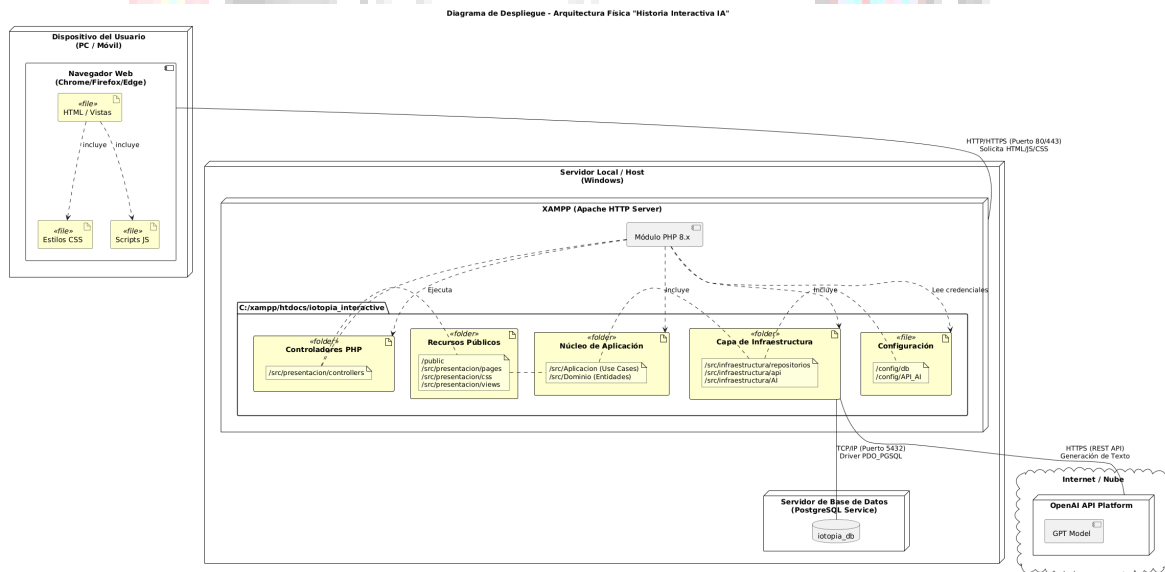
- Fin: Cuando la condición del bucle es No (la historia ha terminado), el Sistema muestra la pantalla final y guarda las estadísticas, concluyendo el flujo de trabajo.

## 4.5. Vista de Despliegue del Sistema "Historia Interactiva IA"

### Gobernado por: Deployment Viewpoint (definido en la Sección 3.5).

Propósito: Esta vista describe la asignación de los componentes de software a la infraestructura de hardware y los entornos de ejecución. Su objetivo es visualizar la topología de la red, los protocolos de comunicación y la distribución física de los artefactos (archivos y bases de datos). Es fundamental para entender cómo se instala, ejecuta y conecta el sistema en un entorno real.

**Modelo: Diagrama de Despliegue** El siguiente diagrama ilustra la arquitectura física del sistema, que opera bajo un esquema híbrido (Servidor Local + Servicios en la Nube).



### Explicación de la Topología y Justificación Arquitectónica

El diagrama revela una arquitectura de 3 Capas (3-Tier Architecture) distribuida, complementada con integración de servicios externos. A continuación se detallan las decisiones de diseño y los patrones aplicados:

- Patrón Cliente-Servidor (Nodos 1 y 2)

- **Funcionamiento:** El sistema divide claramente la responsabilidad entre el "Dispositivo del Usuario" (Cliente) y el "Servidor Local" (Backend).
- **Por qué se eligió:** Esta separación es esencial para la accesibilidad web. El usuario no necesita instalar nada complejo (solo un navegador), mientras que el servidor centraliza la lógica pesada.
- **Distribución de Artefactos:** Observamos que el navegador descarga artefactos estáticos (HTML, CSS, JS), mientras que el servidor retiene los artefactos dinámicos (PHP, Config). Esto optimiza el ancho de banda y protege el código fuente del backend (seguridad por ocultamiento).

## 2. Entorno de Ejecución XAMPP (Apache + PHP)

- **Justificación de Infraestructura:** Se utiliza Apache como servidor web y PHP como módulo de procesamiento.
  - **Resolución de Interoperabilidad:** Apache actúa como un intermediario universal que puede servir la aplicación a cualquier sistema operativo (Windows, Linux, MacOS) a través del protocolo estándar HTTP/HTTPS.
  - **Mapeo de Carpetas:** El diagrama muestra explícitamente cómo la estructura lógica de carpetas (/src/Dominio, /src/infraestructura) se despliega físicamente dentro de htdocs. Esto facilita el despliegue (deployment) mediante simple copia de archivos, resolviendo la preocupación de Facilidad de Instalación.

## 3. Desacoplamiento de la Persistencia (Nodo PostgreSQL)

- **Patrón de Servidor de Base de Datos Independiente:** Aunque en un entorno de desarrollo (XAMPP) la base de datos pueda residir en la misma máquina



física, el diagrama la modela como un nodo de servicio distinto accesible vía TCP/IP (Puerto 5432).

- Por qué funciona así: Esto permite Escalabilidad Vertical. Si el sistema crece, se puede mover la base de datos a un servidor dedicado sin cambiar una sola línea de código (solo ajustando el archivo conexion.php en el artefacto de Configuración).

#### 4. Integración de Servicios en la Nube (Nodo OpenAI)

- Arquitectura Híbrida / SaaS: El sistema delega la generación de narrativa a la nube ("OpenAI API Platform").
- Resolución de Preocupaciones de Hardware: Ejecutar un Modelo de Lenguaje (LLM) localmente requeriría hardware costoso (GPUs potentes) en el servidor. Al usar una API externa vía HTTPS, el sistema reduce drásticamente los requisitos de hardware del servidor local, haciendo el proyecto viable y económico (Eficiencia de Recursos).

Preocupación      Cómo lo resuelve este diagrama

|                    |                                                                                                                                                                                           |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seguridad de Datos | El diagrama muestra que el archivo conexion.php y la carpeta Config residen en el servidor y son consumidos por PHP, nunca expuestos directamente al navegador del cliente.               |
| Rendimiento        | La carga de trabajo se distribuye: el Navegador renderiza la UI, el Servidor procesa la lógica y la Nube genera el texto complejo. Ningún nodo se satura individualmente.                 |
| Mantenibilidad     | La agrupación de artefactos en carpetas físicas (Controllers, CoreLogic, Infra) permite a los desarrolladores saber exactamente dónde ubicar los archivos para actualizaciones o parches. |

Disponibilidad

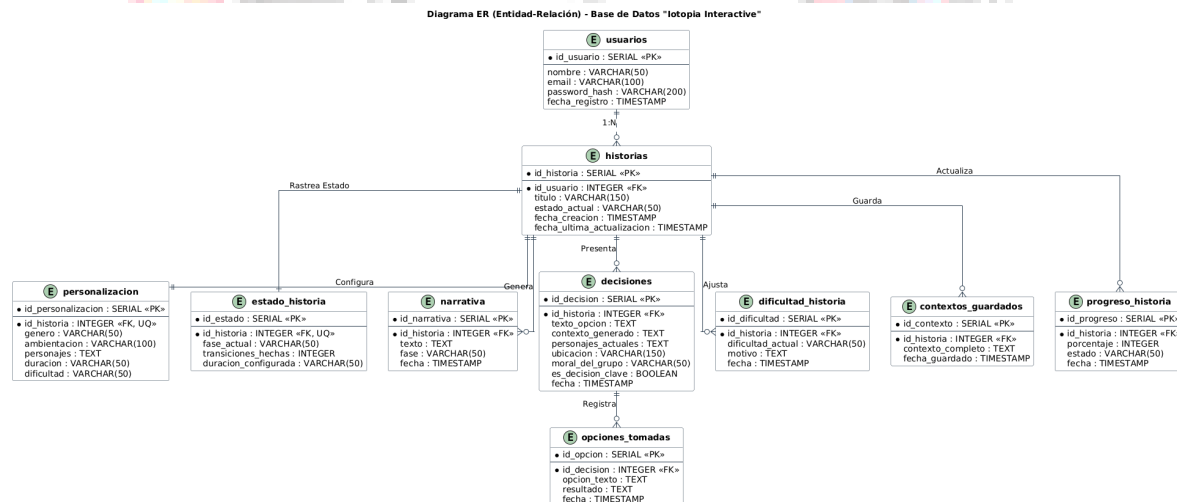
Se evidencian las dependencias de red críticas. Si falla la conexión a Internet, falla el nodo OpenAI. Esto documenta explícitamente los puntos de fallo externos.

## 4.6. Vista de Datos del Sistema "Historia Interactiva IA"

### Gobernado por: Data Viewpoint (definido en la Sección 3.6).

Propósito: Esta vista describe la estructura de información persistente del sistema. Su objetivo es definir los modelos de datos, las relaciones entre entidades y las reglas de integridad que gobiernan el almacenamiento. Dado que la generación de narrativa con IA es inherentemente volátil (se pierde al cerrar la sesión), esta vista es crítica para garantizar la continuidad, coherencia y auditoría de la historia del usuario a largo plazo.

**Modelo: Diagrama Entidad-Relación (ER)** El siguiente diagrama presenta el esquema normalizado implementado en el motor de base de datos.



### 4.6.1. Justificación de Tecnología y Paradigma

Para la capa de persistencia de "Ilotopia Interactiva", se ha seleccionado PostgreSQL bajo un Modelo Relacional (SQL). Esta decisión arquitectónica se fundamenta en los siguientes puntos:

- ¿Por qué un Modelo Relacional (SQL) y no NoSQL?
  - Integridad Referencial Estricta: En una historia interactiva, una "Decisión" no puede existir sin una "Historia", y una "Personalización"

debe estar atada obligatoriamente a una partida. El modelo relacional permite definir claves foráneas (FK) que impiden la creación de datos huérfanos o inconsistentes.

- Estructura Predecible: Aunque el contenido de la historia (el texto generado por la IA) es variable, la estructura del juego (Estado, Dificultad, Configuración) es rígida y bien definida. El esquema SQL fuerza a cumplir estas reglas de negocio desde la base de datos.

## 2. ¿Por qué PostgreSQL?

- Soporte de Texto Robusto: Postgres maneja campos TEXT de gran tamaño de manera eficiente, lo cual es vital para almacenar los párrafos generados por la IA y los contextos completos.
- Fiabilidad y ACID: El sistema realiza operaciones complejas (avanzar estado + guardar decisión + actualizar dificultad). La conformidad ACID de Postgres asegura que estas transacciones sean atómicas: o se guardan todos los cambios o no se guarda ninguno, evitando que una historia quede "corrupta" a mitad de un proceso.
- Escalabilidad Futura: Postgres permite tipos de datos JSONB, lo que ofrece flexibilidad si en el futuro se decide guardar metadatos de la IA semi-estructurados sin romper el esquema relacional.

### 4.6.2. Explicación del Esquema y Resolución de Preocupaciones

El diseño del esquema responde directamente a las necesidades funcionales y a los patrones de arquitectura utilizados:

#### 1. Persistencia del Patrón State (Tabla estado\_historia)

- Diseño: Se ha separado el estado de la tabla principal historias mediante una relación 1:1 con estado\_historia.
- Justificación: Esta tabla actúa como la "memoria" del Patrón State implementado en el código. Campos como fase\_actual y transiciones\_hechas son leídos por las clases de dominio (EstadoInicio, EstadoDesarrollo) para saber cómo comportarse. Sin esta tabla, el sistema olvidaría en qué fase del juego está el usuario tras cerrar el navegador.

## 2. Contexto para la IA (Tablas narrativa, decisiones, contextos\_guardados)

- Resolución de la preocupación "Amnesia de la IA": Los Modelos de Lenguaje (LLM) no tienen memoria a largo plazo.
- Funcionamiento: Las relaciones 1:N entre historias y decisiones/narrativa permiten reconstruir el historial completo. Cuando el usuario retoma una partida, el sistema consulta estas tablas, concatena los últimos eventos y se los envía a la IA como "contexto". La tabla contextos\_guardados funciona como puntos de control (checkpoints) optimizados para no tener que leer toda la historia cada vez.

## 3. Normalización de la Configuración (Tabla personalizacion)

- Diseño: Los datos de configuración inicial (género, ambientación, dificultad base) se extraen a su propia tabla.
- Por qué: Esto cumple con el principio de Separación de Intereses. La tabla historias se mantiene ligera (solo meta-datos), mientras que los detalles inmutables de la configuración residen aparte. Esto optimiza las consultas cuando solo se necesita listar las partidas en el menú principal.

## 4. Auditoría y Adaptabilidad (Tabla dificultad\_historia)

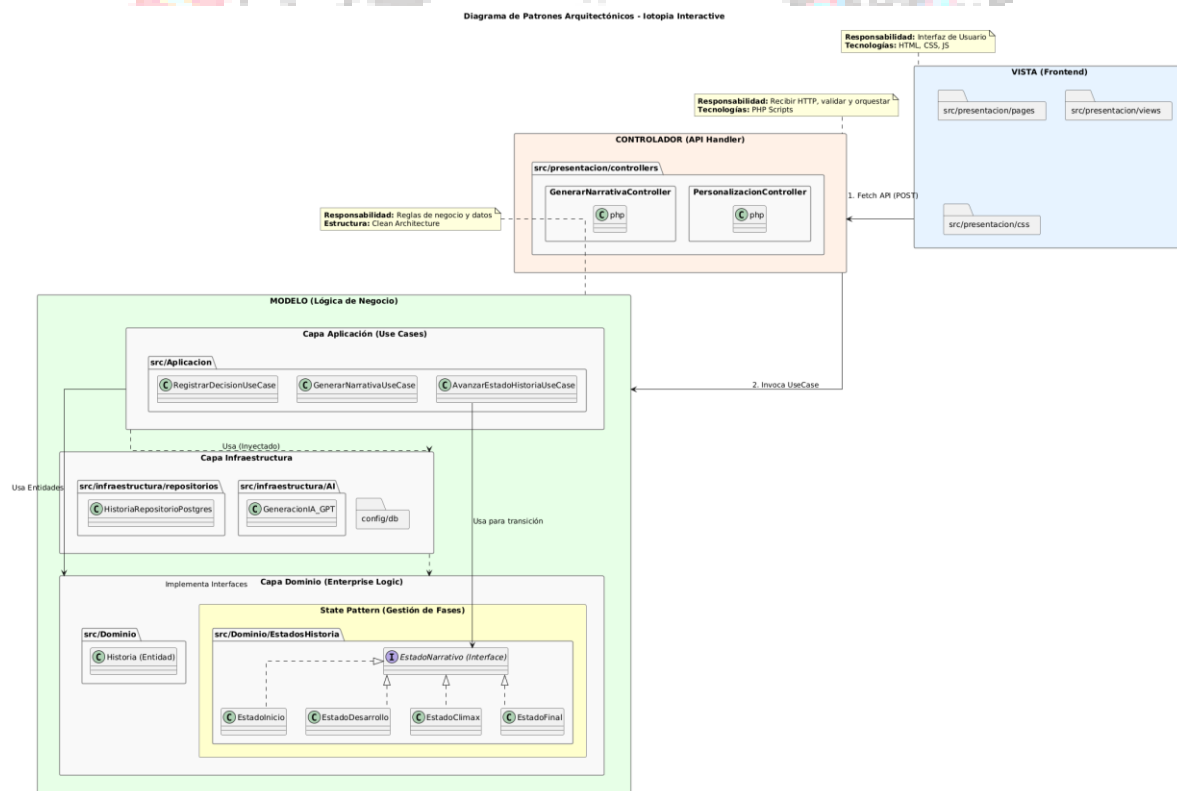
- Diseño: Relación 1:N que registra cada cambio de dificultad y su motivo.
- Por qué: Permite a los desarrolladores o administradores analizar el balance del juego. Se puede trazar una gráfica de cómo la IA ajustó la dificultad basándose en las acciones del usuario, validando si el algoritmo de "Temperatura" funciona correctamente

## 4.7. Vista de Patrones de Diseño y Arquitectura

**Gobernado por: Patterns/Style Viewpoint (definido en la Sección 3.7).**

**Propósito:** Esta vista documenta las decisiones fundamentales de diseño que dan forma al sistema "Ilotopia Interactive". Define los estilos arquitectónicos y los patrones de diseño aplicados para resolver problemas recurrentes como el desacoplamiento, la gestión de estados complejos y la independencia de la tecnología. Su objetivo es justificar la elección de una arquitectura en capas y explicar cómo los patrones seleccionados garantizan la calidad del software.

**Modelo: Diagrama de Patrones Arquitectónicos** El siguiente diagrama ilustra la jerarquía de patrones, desde el nivel macro (MVC) hasta el nivel micro (State Pattern), y cómo interactúan las capas de Clean Architecture.



### 4.7.1. Análisis de Estilos y Patrones Seleccionados

El sistema no es monolítico; se ha construido siguiendo una estricta separación de responsabilidades mediante la integración de tres patrones clave:

## 1. Patrón Arquitectónico Macro: MVC (Model-View-Controller)

- Aplicación en el proyecto:
  - Vista: HTML/JS en el cliente.
  - Controlador: Scripts PHP (GenerarNarrativaController) que actúan como API Handlers.
  - Modelo: Toda la lógica interna encapsulada (Clean Architecture).
- ¿Por qué se eligió? Este patrón resuelve la preocupación de Interoperabilidad. Al desacoplar la Vista del Modelo mediante una API (Controladores), el sistema permite que el frontend evolucione independientemente del backend. Si en el futuro se decide cambiar la interfaz web por una App Móvil, la lógica del Modelo y los Controladores permanecen intactos.

2. Arquitectura Interna: Clean Architecture (Arquitectura Limpia) El "Modelo" del MVC no es una masa de código desordenada, sino que se estructura en 3 capas concéntricas siguiendo el Principio de Dependencia (las capas externas dependen de las internas, nunca al revés):

- Capa 1: Dominio (Núcleo):
  - Componentes: Entidades (Historia, Decision) e Interfaces.
  - Justificación: Es el corazón del sistema. Aquí residen las reglas de negocio que no cambian aunque cambie la tecnología. Es independiente de la base de datos y de la IA.
- Capa 2: Aplicación (Casos de Uso):
  - Componentes: Clases UseCase (GenerarNarrativaUseCase).
  - Justificación: Orquesta el flujo de datos. Define "qué puede hacer el usuario". Actúa como intermediario entre la infraestructura y el dominio.

- Capa 3: Infraestructura (Detalles):
  - Componentes: Repositorios Postgres e Integración GPT.
  - Justificación: Resuelve la preocupación de Independencia Tecnológica. Si OpenAI deja de funcionar o se cambia Postgres por MySQL, solo se modifica esta capa externa. El núcleo del juego (Dominio) no se ve afectado.

### 3. Patrón de Comportamiento: State Pattern (Patrón Estado)

- Aplicación: Implementado en el Dominio (EstadoInicio, EstadoDesarrollo, etc.).
- ¿Por qué es esencial para este proyecto? Una historia interactiva es, por definición, una máquina de estados finitos. Gestionar la transición entre "Inicio", "Desarrollo" y "Final" mediante condicionales (if/else) resultaría en un código frágil y difícil de mantener ("Spaghetti Code").
- Beneficio: El patrón State encapsula el comportamiento de cada fase en su propia clase.
  - Ejemplo: El EstadoInicio sabe que debe transicionar a EstadoDesarrollo después de 2 decisiones.
  - Ejemplo: El EstadoFinal sabe que ya no debe permitir generar más narrativa. Esto garantiza la Integridad Lógica del juego: es imposible saltar reglas narrativas accidentalmente.

#### 4.7.2. Justificación Global: ¿Por qué esta es la mejor manera de construir el sistema?

La combinación de MVC + Clean Architecture + State Pattern se seleccionó para resolver las preocupaciones críticas de los stakeholders académicos y técnicos:

1. Mantenibilidad a Largo Plazo: Al separar las capas, un error en la consulta SQL (Infraestructura) nunca romperá las reglas de negocio de la historia (Dominio). El código es modular y fácil de depurar.

2. Testabilidad (Testing): Clean Architecture permite probar los Casos de Uso de forma aislada. Podemos simular (Mock) la IA y la Base de Datos para probar si la lógica del juego funciona, sin necesidad de gastar créditos en la API de OpenAI o tener el servidor SQL encendido.
3. Adaptabilidad: El uso de Interfaces en el Dominio permite cambiar la "Inteligencia" del sistema. Hoy usamos GPT-4, mañana podríamos conectar un modelo local Llama-3 simplemente creando una nueva clase en Infraestructura que implemente la misma interfaz, sin tocar el resto del código.

## 5. Coherencia y Reglas de Correspondencia (Cláusula 5.7)

Esta sección define las reglas que aseguran la consistencia y trazabilidad entre las diferentes vistas arquitectónicas (Contexto, Composición, Lógica, Interacción, Despliegue, Datos y Patrones). Su objetivo es garantizar que las vistas no se contradigan entre sí y que el diseño se mantenga íntegro a medida que evoluciona

### 5.7.2 Reglas de Correspondencia

Las siguientes reglas se deben cumplir en toda la documentación de la arquitectura:

- Regla 1 (Contexto -> Composición): Toda interacción con un actor externo identificada en la Vista de Contexto (ej. Usuario Final, Motor de IA) debe ser manejada por un componente específico en la Vista de Composición (ej. Interfaz de Usuario, API Gateway).
- Regla 2 (Composición -> Lógica): Todo componente de software en la Vista de Composición (ej. GeneracionIA, Autenticacion) debe tener una o más



clases que implementen su funcionalidad principal en la Vista Lógica (ej. las clases GeneracionIA, Autenticacion).

- Regla 3 (Lógica -> Datos): Toda entidad de datos (clases en el paquete de Dominio) que requiera persistencia en la Vista Lógica (ej. Usuario, Historia, Decision) debe tener una representación correspondiente como una o más tablas en la Vista de Datos (ERD) (ej. usuarios, historias, decisiones).
- Regla 4 (Lógica -> Interacción): Todo método público principal de un servicio en la Vista Lógica (ej. GeneracionIA.elegirOpcion()) debe ser mostrado como parte de un flujo de mensajes en al menos un Diagrama de Secuencia en la Vista de Interacción.
- Regla 5 (Composición -> Despliegue): Todo componente de software en la Vista de Composición (ej. Interfaz de Usuario, API Gateway) debe mapearse a un artefacto de software (.js/.html, .php) en la Vista de Despliegue.
- Regla 6 (Despliegue -> Contexto): Toda conexión de red entre nodos en la Vista de Despliegue (ej. Cliente <-> Servidor XAMPP) debe corresponder a un flujo de datos identificado en la Vista de Contexto.
- Regla 7 (Patrones -> Todas las Vistas): La estructura de clases en la Vista Lógica y la organización de componentes en la Vista de Composición deben adherirse estrictamente a las capas y patrones definidos en la Vista de Patrones Arquitectónicos (MVC, Clean, State).

### 5.7.1 Inconsistencias Conocidas

Si bien se busca una consistencia total, es normal que durante el desarrollo existan incongruencias temporales. A la fecha de esta versión del documento, se registran las siguientes:

- Inconsistencia 1 (Lógica/Datos): La clase GuardarPersonalizacion en la Vista Lógica no tiene una tabla de persistencia directa en la Vista de Datos. La lógica actual fusiona sus atributos en la tabla personalizaciones, pero la clase de servicio no refleja esta operación de guardado, solo de configuración.
- Inconsistencia 2 (Interacción/Lógica): Los flujos de manejo de errores (ej. ¿qué pasa si la GeneracionIA falla o la Base de Datos no responde?) no están modelados en los Diagramas de Secuencia de la Vista de Interacción.
- Inconsistencia 3 (Lógica): La clase ObtenerEstadisticas no tiene un método explícito para consultar la base de datos; su dependencia de GestionarHistoria está implícita pero no definida en los métodos de la clase en la Vista Lógica.

## 6. Justificación de las Decisiones Arquitectónicas (Architecture Rationale)

### Propósito:

Esta sección documenta la evidencia y el razonamiento utilizado para seleccionar la arquitectura actual del sistema "Historia Interactiva IA". Se detallan las alternativas consideradas, las decisiones tomadas y cómo estas satisfacen los requisitos críticos de los interesados (stakeholders), equilibrando las limitaciones técnicas con los objetivos de calidad.

### 6.1. Decisión Fundamental: Adopción de Clean Architecture y MVC

- El Problema: El sistema combina lógica de juego compleja (reglas de estado), integración con servicios inestables (IA externa) e interfaces de usuario web. Mezclar estos dominios en scripts monolíticos crearía un sistema frágil ("Spaghetti Code") difícil de probar y modificar.
- La Decisión: Implementar una arquitectura en capas estricta: Clean Architecture para el backend, envuelta en un patrón MVC para la entrega web.
- Justificación:

1. Protección del Núcleo: La lógica del juego (Entidades y Reglas) se aísla en el centro. Si la API de OpenAI cambia o si migramos de PostgreSQL a MySQL, las reglas del juego no se tocan.
2. Testabilidad: Permite probar la lógica narrativa usando "Mocks" (simuladores) de la IA y la Base de Datos, facilitando el desarrollo sin costes de API constantes.

- Preocupaciones Resueltas: Mantenibilidad (Facilidad de cambio) y Evolucionabilidad.

## 6.2. Decisión de Diseño: Implementación del Patrón State

- El Problema: La narrativa interactiva tiene un flujo estricto (Inicio  $\rightarrow$  Desarrollo  $\rightarrow$  Clímax  $\rightarrow$  Final). Gestionar estas transiciones y sus condiciones (ej. "no pasar al clímax si no hay 5 decisiones") mediante condicionales if/else anidados es propenso a errores lógicos y difícil de escalar.
- La Decisión: Utilizar el Patrón de Diseño State (Estado). Cada fase de la historia es una clase independiente (EstadoInicio, EstadoDesarrollo) que encapsula sus propias reglas de transición.
- Justificación:
  1. Integridad Lógica: Garantiza que el sistema nunca entre en un estado inválido. La clase Historia delega el comportamiento al estado actual, eliminando la complejidad ciclomática de los controladores.
  2. Escalabilidad Narrativa: Si en el futuro se desea agregar un "Epílogo" o un "Capítulo Secreto", solo se crea una nueva clase de Estado sin modificar el código existente (Principio Open/Closed).
- Preocupaciones Resueltas: Corrección Funcional y Extensibilidad.

### 6.3. Decisión de Persistencia: PostgreSQL y Modelo Relacional

- El Problema: La IA no tiene memoria a largo plazo. El sistema necesita reconstruir el contexto completo de la historia en cada interacción, garantizando que no se pierdan datos entre decisiones.
- La Decisión: Utilizar PostgreSQL con un esquema relacional normalizado.
- Justificación:
  1. Integridad Referencial: A diferencia de una base de datos NoSQL, el modelo relacional impide la existencia de decisiones huérfanas (sin historia asociada).
  2. Transaccionalidad (ACID): El proceso de "Registrar Decisión" implica tres escrituras simultáneas: guardar la decisión, actualizar el estado de la historia y registrar el cambio de dificultad. PostgreSQL garantiza que estas tres operaciones ocurran atómicamente; si una falla, todo se revierte, evitando partidas corruptas.
  3. Manejo de Texto: PostgreSQL está altamente optimizado para manejar grandes volúmenes de texto (TEXT), esencial para almacenar la narrativa generada.
- Preocupaciones Resueltas: Integridad de Datos y Fiabilidad.

### 6.4. Decisión de Infraestructura: IA como Servicio (SaaS) vs. Modelo Local

- El Problema: La generación de historias requiere Modelos de Lenguaje (LLM) de gran tamaño.
- Alternativas:
  - Opción A: Alojarse un modelo (ej. Llama-3) en el servidor local.
  - Opción B: Consumir una API externa (OpenAI GPT).
- La Decisión: Utilizar la Opción B (API Externa) a través de una interfaz adaptadora (GeneracionIA\_GPT).
- Justificación:
  1. Viabilidad Económica: Ejecutar un modelo local con la calidad requerida demandaría servidores con GPUs costosas (VRAM alta). La

API permite un modelo de pago por uso, ideal para un proyecto académico/indie.

2. Calidad Narrativa: Los modelos comerciales actuales (GPT-4) superan a los modelos locales ligeros en coherencia narrativa y seguimiento de instrucciones complejas (JSON mode).

- Preocupaciones Resueltas: Rendimiento y Costo de Infraestructura.

## 6.5. Tabla de Análisis de Compromisos (Trade-offs)

Toda decisión arquitectónica conlleva un sacrificio. A continuación, se reconocen los compromisos asumidos:

| Decisión                      | Ventaja Principal (Pros)                   | Compromiso / Desventaja (Cons)                                | Mitigación Aplicada                                                                     |
|-------------------------------|--------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Clean Architecture            | Código altamente organizado y desacoplado. | Aumenta el número de archivos y la complejidad inicial.       | Uso de inyección de dependencias para simplificar la conexión entre capas.              |
| IA Externa (API)              | Calidad superior sin hardware costoso.     | Latencia de red y dependencia de un tercero.                  | Implementación de GeneradorSimulado para pruebas y manejo de errores (circuit breaker). |
| Patrón State                  | Lógica de transición robusta y clara.      | Más clases para gestionar que un simple switch.               | Diagramas UML claros para documentar el flujo de estados.                               |
| Renderizado en Servidor (PHP) | Simplicidad y seguridad del código fuente. | Mayor carga en el servidor comparado con una SPA (React/Vue). | Uso de AJAX (fetch) para actualizaciones parciales sin recargar toda la página.         |

## Conclusión

La arquitectura de "Itopia Interactive" prioriza la integridad de los datos y la mantenibilidad sobre la velocidad de desarrollo inicial. Se ha construido un sistema robusto capaz de gestionar la imprevisibilidad de la IA dentro de un marco de reglas de juego estricto, cumpliendo así con los requisitos académicos y funcionales del proyecto.

